A DECLARATIVE SYSTEM FOR OPTIMIZING AI WORKLOADS

A PREPRINT

Chunwei Liu*, Matthew Russo*, Michael Cafarella,
Lei Cao†, Peter Baille Chen, Zui Chen, Mike Franklin‡,
Tim Kraska, Samuel Madden, Gerardo Vitagliano

MIT, †University of Arizona, ‡University of Chicago

chunwei@mit.edu, mdrusso@mit.edu, michjc@csail.mit.edu
caolei@arizona.edu, peterbc@mit.edu, chenz429@mit.edu, mjfranklin@uchicago.edu,
kraska@mit.edu, madden@csail.mit.edu, gerarvit@csail.mit.edu

ABSTRACT

Modern AI models provide the key to a long-standing dream: processing analytical queries about almost any kind of data. Until recently, it was difficult and expensive to extract facts from company documents, data from scientific papers, or insights from image and video corpora. Today's models can often accomplish these tasks with high accuracy. However, a programmer who wants to answer a substantive AI-powered query must orchestrate large numbers of models, prompts, and data operations. For even a single query, the programmer has to make a vast number of decisions such as the choice of model, the right inference method, the most cost-effective inference hardware, the fastest and most accurate prompt formulation, and so on. The optimal set of decisions can change as the query changes and as the rapidly-evolving technical landscape changes. In this paper we present PALIMPZEST, a system that enables anyone to process AI-powered analytical queries simply by defining them in a declarative language. The system then uses its cost optimization framework — which explores the search space of AI models, prompting techniques, and related foundation model optimizations — to implement the query with the best trade-offs between runtime, financial cost, and output data quality. We describe the workload of AI-powered analytics tasks, the optimization methods that PALIMPZEST uses, and the prototype system itself. We evaluate PALIMPZEST on tasks in Legal Discovery, Real Estate Search, and Medical Schema Matching. We show that even our simple prototype offers a range of appealing plans, including a plan that is 3.3x faster, 2.9x cheaper, and offers better data quality than the baseline method. With parallelism enabled, PALIMPZEST can produce plans with up to a 90.3x speedup at 9.1x lower cost relative to a single-threaded GPT-4 baseline, while obtaining an F1-score within 83.5% of the baseline. These require no additional work by the user.

Keywords Relational optimization · LLMs · AI programming

* indicates equal first author contribution.

1 Introduction

Advances in AI models have driven progress in applications such as question answering [62], chatbots [12], autonomous agents [44, 49], and code synthesis [32, 23]. In many cases these systems have evolved far beyond posing a simple question to a chat model: they are complex AI systems[61] that combine elements of data processing, such as Retrieval Augmented Generation (RAG); ensembles of different models; and multi-step chain-of-thought reasoning.

It is easy for the runtime, cost, and complexity of these applications to escalate quickly. Consider a few simple AI-powered analytical questions:

- **Legal Discovery (Figure 8)** — Prosecutors conducting an investigation wish to identify emails which are (a) related to corporate fraud (e.g., by mentioning a specific fraudulent investment vehicle) and (b) not quoting

from a news article reporting on the business in question. Test (a) may be implemented using a regular expression or UDF, while (b) requires semantic understanding to distinguish between employees sharing news articles versus first-hand sources of information. An efficient implementation would recognize that (a) can likely be implemented using conventional and inexpensive methods, while (b) may require an LLM or newly-trained text model to retain good quality.

- **Real Estate Search (Figure 2)** — A homebuyer wants to use online real estate listing data to find a place that is (a) modern and attractive, and (b) within two miles of work. Test (a) is a semantic search task that involves analyzing images, while (b) is a more traditional distance calculation over extracted geographic data. Any implementation needs to process a large number of images and listings, limit its use of slow and expensive models, and still obtain high-quality results.
- **Medical Schema Matching (Figure 9)** — In the medical domain, cancer research is often based on large collections of information about patient cases and sample data. However, research studies' data outputs do not always conform to a unified standard. Based on the medical data pipeline described by Li, *et al.* [31], we imagine a researcher who would like to (a) download the datasets associated with a dozen specified cancer research papers, (b) identify the datasets that contain patient experiment data, and (c) integrate those datasets into a single table. Step (a) requires parsing and understanding research paper texts to obtain dataset URLs, step (b) requires classifying each dataset as either patient-related or not, and step (c) requires a complex data integration task. As with the use cases above, the programmer must manage multiple subtasks, each of which offer different possible optimization opportunities and quality trade-offs.

These tasks:

1. Interleave traditional data processing with AI-like semantic reasoning
2. Are data-intensive: each source dataset could reasonably range from hundreds to millions of records
3. Can be decomposed into an execution tree of distinct operations over sets of records

Semantic Analytical Applications: Taken together, these criteria outline a broad class of AI programs that are important, complex, and potentially very optimizable; we call them *semantic analytical applications* — or, **SAPPs**. We believe there is a large set of such use cases that mix conventional data analytics with transformations and filters that would not be possible without AI methods. Such workloads frequently require interleaved data acquisition steps, conventional analytical queries, and AI operations. The AI operations process unstructured data, require broad domain knowledge to implement, or have specifications that users may not be able to implement correctly with traditional source code.

Challenges: Naively scaling SAPPs to process thousands or millions of inputs appears to require spending a huge amount of runtime and money executing high-end AI models. The performance gap between traditional data processing components and AI-powered components is profound. For example, a high-quality open-source LLM running on a modern GPU might process 100-125 tokens per second. Assuming a token is represented by 5 bytes (on average), such a model yields a throughput of *less than 1 KB per second.* OpenAI's new GPT-4o model currently costs 5 USD for 1M input tokens, or in other words 5 USD for processing just 5MB of data. These numbers are many orders of magnitude worse than any other component of the modern data processing stack, such as data storage, network bandwidth, SQL query processing time, and so on.

Optimizing use of AI components seems crucial, while at the same time the AI infrastructure is in incredible technical flux. New models, and implementation techniques are published seemingly every day, while model costs and runtimes change constantly. Harnessing the latest advances in model runtime, cost, and quality appears to require engineers to constantly rewrite and retune their systems.

Consider the wide range of technical decisions the AI engineer faces:

- When designing prompts, the engineer must optimize wording, choose to employ zero- or few-shot examples, and perhaps decide on a general prompting strategy (e.g., chain-of-thought, ReAct [59], etc.).
- When choosing models, the engineer must pick the best model *for each substask in the program*, balancing time, cost, and quality.
- When running the workload, the engineer must decide whether each subtask is best implemented by a foundation model query, synthesized code, or a locally-trained student model. Furthermore, they must consider how to combine tasks to improve GPU cache utilization, and how to avoid running over LLM context limits.
- When scaling out to a larger dataset, the engineer faces additional challenges in selecting an efficient execution plan. Even if the system performs well on a small dataset, it may require redesign to ensure reasonable

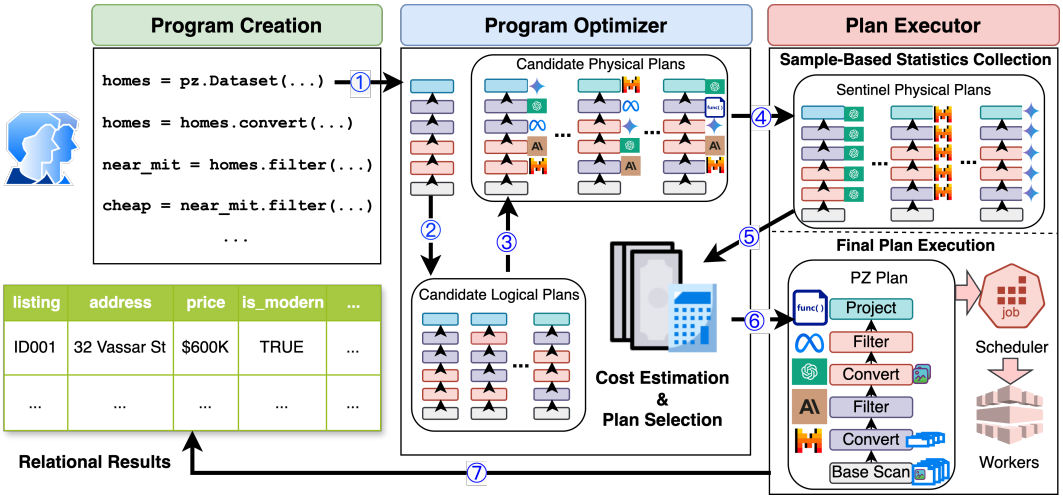


Figure 1: Overview of the PALIMPZEST system. Users write their program(s) in a declarative language which undergo compilation ①, logical plan generation ②, and physical plan generation ③ . Subsequent steps involve profiling sample plans ④ and analyzing performance statistics to estimate costs ⑤. The optimal plan, tailored to user-specified preferences (e.g. to maximize quality at fixed cost), is selected ⑥ and executed, delivering relational results to the user ⑦. This comprehensive process is designed to optimize execution by effectively balancing cost, runtime, and quality.

runtime, cost, and performance at a larger scale. This may involve enabling parallelism for each component and integrating them seamlessly into the broader system for optimal efficiency.

- When integrating with external data systems, the engineer must decide how to choose parameters (e.g., the number of chunks to return per RAG query) in a manner that yields the best speed, cost, and quality trade-offs.

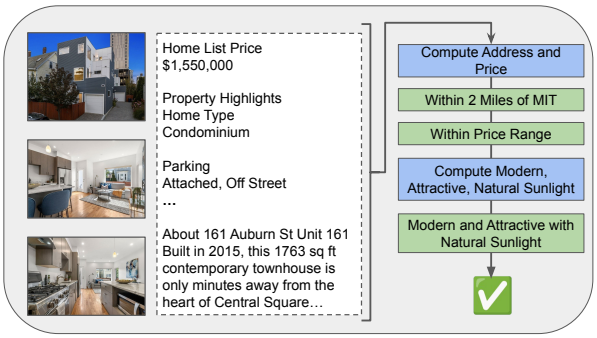
The space of possible decisions is vast, and choosing wisely depends on low-level details of the exact task being performed. Moreover, the definition of "best" can change over time: the best program might be "fast and cheap" during initial development, then "costly but high-quality" during final deployment. Finally, the changing technical landscape means that optimization choices made today might be obsolete tomorrow.

Our Goal: The key insight is that machines, not human engineers, should decide how best to optimize semantic analytics applications. Engineers should be able to write AI programs at a high level of abstraction and rely on the computer to find an optimized implementation that best fits their use case. A similar set of circumstances — a need for performance improvements for an important workload, during a time of enormous technical change — led to the development of the relational database query optimizer in the 1970s. Today's underlying technical challenges are very different, but the basic idea of declarative program optimization remains valuable.

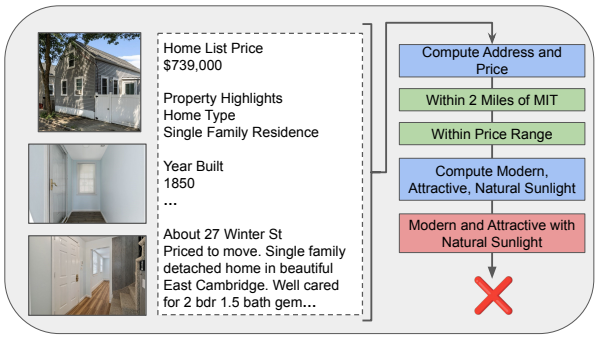
In this paper we lay out our vision and a prototype for PALIMPZEST[1], a system that enables engineers to write succinct, declarative code that can be compiled into optimized programs. PALIMPZEST is designed to optimize the broad SAPP workload class, which should encompass large-scale information extraction, data integration, discovery from scientific papers, image understanding tasks, and multimodal analytics. When running an input user program, PALIMPZEST considers a range of logical and physical optimizations, then yields a set of possible concrete executable programs. PALIMPZEST estimates the cost, time, and quality of each one, then chooses a program based on runtime user preferences. The system is designed to be extensible, so that new optimizations can be easily added in the future. Just as the RDBMS allowed users to write database queries more quickly and correctly than they could by writing traditional code, PALIMPZEST will allow engineers to write better AI programs more quickly than they could unaided.

Our Approach: A core challenge in building PALIMPZEST is creating an optimizer that can marshal many optimizations to meet a user's cost, runtime, and quality goals. By using a language that is high-level, type-focused, and declarative — rather than the low-level prompting and coding method pursued by naive programming and some other frameworks [15, 64] — we believe PALIMPZEST can exploit many optimizations that are not otherwise available. Another key challenge involves designing a programming interface which simultaneously enables engineers to express the broadest possible set of AI programs, while imposing structure on their programs that the optimizer can exploit. To this end, we created a Python library which implements a thin abstraction over an underlying relational algebra. The core intellectual difference between PALIMPZEST and previous database-style systems is the addition of the relational **convert** operator,

[1]Like an ancient palimpsest, our system entails constant revision and rethinking, in our case by the optimizer. Only zestier!



(a) Example of a positive entry in the Real Estate Search workload. The house meets the three criteria of being (1) close to MIT (2) within the user's price range and (3) modern and attractive with lots of natural sunlight.



(b) Example of a negative entry in the Real Estate Search workload. This house does not meet the criteria set by the user's search because the house is not considered to be modern and attractive.

Figure 2: A positive and negative example in the Real Estate Search workload.

which transforms an object of one user-defined schema to another. This operator — which is implemented using a variety of methods, often based on foundation models — allows the programmer to implement many AI tasks in a relational and optimizable style.

Contributions: In this paper we:

- Introduce Semantic Analytics Applications (SAPPs), a new class of workloads in a new domain that can nonetheless benefit from many traditional ideas in data management. Addressing them requires a range of new technical solutions and abstractions. (Section 2.)
- Describe the PALIMPZEST architecture and how it aims to support data intensive AI workloads. (Section 3.)
- Describe a set of physical and logical optimizations, three of which are implemented in our prototype. (Section 4.)
- Present experimental results that show that even with just these basic optimizations in place, PALIMPZEST can execute SAPP workloads with a range of tradeoffs that are more appealing than a baseline approach. The exact benefit depends on the workload and whether the user prefers to reduce time, reduce cost, or maximize quality. Our results include a plan that is 3.3x faster, 2.9x cheaper, and offers better data quality than its baseline method; and another plan that is 4.7x faster and 9.1x cheaper, with a trade-off of 14.3% lower quality than its baseline. (Section 5.)
- Demonstrate that our prototype can produce parallelized plan implementations which are up to 90.3x faster and 9.1x cheaper than a single-threaded GPT-4 baseline, with an F1-score within 83.5% of the baseline. (Section 5.)

PALIMPZEST is an exciting prototype system and we are looking forward to the potential for future optimizations and features. We invite interested readers to experiment with our code at `https://github.com/mitdbg/palimpzest` and perhaps make a contribution.

2 Workloads

Before we describe the details of the PALIMPZEST system, it is useful to discuss the workloads PALIMPZEST aims to support, in particular the SAPP workloads.

As discussed in Section 1, SAPPs (1) combine traditional data processing and AI elements, (2) potentially process enormous amounts of data, and (3) can be decomposed into a tree of operations over sets of data objects. As a running example, consider the Real Estate Search task in Figure 2. In this task, the user wants to search all of the real estate listings near Cambridge, MA to find a house that is (a) modern and attractive, (b) within two miles of MIT, and (c) within the user's price range.

This task clearly satisfies the first criterion: determining whether a house is "modern and attractive" likely requires using some vision or text model to process the textual listings and images. Moreover, depending on the dataset, recovering price and location data may require extraction from text. The second criterion is easily satisfied because — as of this

writing — Zillow shows 1,327 listings for homes in the Cambridge and Boston areas, and each listing contains a text description along with (typically) 10 or more images. As for the third criterion, the query clearly has a set of objects as both input and output. It is easy to see how we might first have an operator that extracts price and location fields from the raw data, then a second that applies traditional selection predicates. Finally, the system might have an operator that extracts whether the images show a "modern and attractive" residence. Figure 2 shows a conceptual pipeline, with imperative steps listed in blue, satisfied logical tests in green, and failed logical tests in red.

Optimization Challenges. The ideal implementation of an AI system for a SAPP workload will jointly optimize its semantic and conventional elements. For example, in Real Estate Search, an extremely naive implementation might waste time and money processing images of apartments to test whether they are "modern and attractive" — only to discard them when they fail to meet conventional constraints. A slightly more sophisticated implementation would reorder the plan so that it applies the restrictive conventional constraints first, thereby avoiding the time and expense of invoking a vision model on candidates that will later be discarded. Further optimizations might process the text description first to evaluate whether an apartment is likely to be "modern and attractive", and then apply an image processing method only when deemed necessary. The decisions around which optimizations to use and when to use them will have to be made for each new task and dataset.

At its core, optimizing an AI system to execute a given SAPP workload requires making accurate predictions about the runtime, cost, and quality of each semantic and conventional data processing step. Estimating these metrics for semantic tasks can be particularly challenging. For example, estimating the runtime and cost for a vision model requires knowing the average number of input and output tokens per record as well as the total number of records that will be processed by the model (i.e., its cardinality). Estimating the quality of an output — especially without labelled data — may require using heuristics that can be error prone, or comparing against an expensive "champion" model. Finally, for every physical optimization available to the system (such as using an ensemble of vision models or decreasing the image resolution), the optimizer needs to predict the optimization's impact on these metrics.

System Design Challenges. The ideal implementation of a system for processing SAPP workloads will also require minimal engineering effort to maintain as use-case needs and input data change over time. To the extent that AI components have been integrated into data applications in the past, significant effort went into training models narrowly tailored for a particular semantic task. Outside of a few tech giants, models were infrequently retrained. Models were restrictive enough that the space of system-wide performance tuning decisions was not large, and at any rate models changed infrequently enough that system-wide tuning did not have to happen often.

The modern AI landscape has changed all of these assumptions. The set of models and adjacent AI approaches (such as prompting strategy) is large and rapidly changing. The space of system-wide performance decisions is vast, and the best decision changes with each new model, task, business need, and dataset. Anecdotally, we have heard several RAG users and vendors start to complain that deploying these systems requires making repeated configuration decisions that reflect reasonable performance vs quality trade-offs; this is a task that traditional data administrators are often not equipped to make. PALIMPZEST aims to enable AI system engineers to focus on programming the system logic while letting the optimizer select which models and physical optimizations are needed to meet the user's preferences for cost, runtime, or data quality.

3 Overview

We present an overview of the PALIMPZEST system and discuss its primary components. First, we describe PALIMPZEST's relational model, the convert operator, and how AI workloads can be interpreted as computing relational views. Then we discuss a short sample program and language features. Finally, we provide a high-level view of the system's architecture, including walking through a simple program execution.

3.1 Semantic Analytics and the Relational Model

PALIMPZEST treats its programs primarily as a form of computing *relational views*: the user specifies a (set of) input relation(s) (called `Datasets`) and a target output relation to be computed. Each relation has a corresponding `Schema`. The user also describes a series of operations to be applied that transform the inputs into the output.

Figure 3 shows a short example program, which we used for our evaluation in Section 5. In this program, the user wants to identify emails that are not quoting from sources outside of Enron and that reference fraudulent investment vehicles. The first step to use PALIMPZEST is therefore to create a custom schema for the input dataset of Emails – on lines 3-6. In this case, Email is a subclass of `TextFile`, which is defined in PALIMPZEST's core library and inherits directly from the base `Schema` class. Starting on line 9, the user begins to describe data processing actions, beginning with instantiating an initial `Dataset` and declaring it to be following the `Email` schema. The `source`

```
1  import palimpzest as pz
2
3  class Email(pz.TextFile):
4  """Represents an email, which can subclass a text file"""
5    sender = pz.Field(desc="The email address of the sender", required=True)
6    subject = pz.Field(desc="The subject of the email", required=True)
7
8  # define logical plan
9  emails = pz.Dataset(source="enron-emails", schema=Email)
10 emails = emails.filter("The email is not quoting from a news article or an article ...")
11 emails = emails.filter("The email refers to a fraudulent scheme (i.e., \"Raptor\", ...")
12
13 # user specified policy
14 policy = pz.MinimizeCostAtFixedQuality(min_quality=0.8)
15
16 # execute plan
17 results = pz.Execute(emails, policy=policy)
```

Figure 3: The AI program written using PALIMPZEST for the Legal Discovery workload.

string “enron-emails" uniquely identifies a set of files that have been preregistered with PALIMPZEST (see Subsection 3.5 for more details). The code on line 9 transforms the raw input data objects into the `Email` Schema and inserts the results into the `emails` Dataset. On line 10, the program filters `emails` for the subset which are not quoting from news articles. On line 11 the program filters for emails which discuss fraudulent investment entities.

The programmer takes two more steps: on line 14, she specifies a `policy` that describes how the system should choose among multiple possible implementations of the steps described so far. (In this case, the plan with the lowest expected financial cost, subject to a lower bound on quality, is preferred.) Finally, on line 17, the programmer asks PALIMPZEST to `Execute()` the program; this entails generating a logical execution plan, generating multiple optimized physical execution plan candidates, choosing one according to the specified policy, and then executing the code and yielding results. Programs written with PALIMPZEST are executed lazily, thus no actual data processing occurs until line 17.

Unlike SQL, PALIMPZEST is intended mainly to be used as a library in a host language; although the current implementation is in Python, there is no reliance on unique features of the Python language, and porting the system to other languages would be straightforward. Even a host-independent program syntax, akin to SQL, would be possible. However, since we view easy programmatic integration with other data processing elements as a core design goal, we think that a strong hosted-language system is a better design choice.

Figure 4b shows the logical relational operators supported by PALIMPZEST. Some of the operators, such as groupby, aggregation, and limit, are not showcased in Figure 3, but follow their standard definitions from the data management literature. Line 9 — which creates the initial `Dataset` — implies a new operator: the **Convert** operation. This operator transforms a record of one schema into a record of another schema. In this case, the source data objects have a default schema of `File`, which must be converted to examples of `Email`. This operator is how PALIMPZEST implements most of its AI-intensive steps and is worth describing in detail.

3.2 Convert

Convert transforms a typed data object into a new object with a specified schema. The name may suggest it is tailored for a narrow model management task, such as converting an operational schema into a star schema. But we take a very broad view of what the convert operator should be able to do; the name is meant to describe the task of converting one arbitrary type into another.

Executing a particular convert operator may entail textual information extraction, text summarization, classification, image understanding, or any number of other AI tasks. The correct behavior of a convert operation is in most cases implied only by the user’s specification of the input and output `Schemas`. In the case of Figure 3, the convert operator must extract the `sender` and `subject` fields from a raw data object in order to produce an `Email`.

Here are a few examples of useful convert tasks, from simple to more complex:

- The system might convert a `TextFile` into an **Email** by extracting the email’s sender and subject (Figure 3).
- It can convert a `PDFFile` into a **ScientificPaper** by extracting the title, authors, abstract, and citations if the content includes elements of a scientific paper, returning None otherwise. (Figure 4a).
- It can convert an `Image` into a **DogBreed** by extracting the breed of any dogs that are present and returning None otherwise. (Figure 4a).

Crucially, **the user does not need to specify how to implement a particular convert operation**. Instead, it is the system's job to implement and perform the operation. By employing a range of different AI models and generation techniques, PALIMPZEST can automatically compute the conversion function for each pair of schemas observed in the user's program. (In some cases, users may wish to hard-code convert operations using lambda expressions for guaranteed conversions, but this is not required and — in our experience — rarely necessary.)

In some cases, the user might specify a convert task that the system cannot figure out how to perform, either because the operator is not implemented well, or because the user's conversion request makes no sense. That is fine: the operator is allowed to silently fail for any element in a `Dataset`, in which case the input object is simply dropped. When converting a Dataset of schema A to schema B, the resulting set may contain fewer records than the original (e.g., if some records are dropped). It is also possible for the resulting set to contain more records than the original (e.g., if the conversion extracts each author of a scientific paper into a separate record).

We think the convert operator will be especially useful when the input object comprises multiple low-level datatypes, such as a real estate listing that has information in both image and text. Many listings will contain price data in text, but some include price directly embedded inside the images. Ideally, the user could forget about carefully specifying exactly which real estate subfield holds the crucial information; instead, the system will simply figure it out. At the moment, implementing this flexibility is future work.

The implementation of the convert operator is intentionally abstracted from the user, as its internal mechanics can vary between executions and may not be representable in a standard transformation language due to the diverse input and output schemas it handles. This operator accepts data in one schema and outputs it in another, adapting dynamically to changing execution conditions. While our current system often uses LLM inference to implement convert, the user cannot count on it. However, implementation details might be accessible through administrator-style tools.

The convert operator is meant to transform data instances — not models — but it has similarities to some concepts from the model management literature. It is close to an execution of the COMPLEXMATCH operator described by Bernstein, followed by an execution of the resulting mapping on a particular data record [7]. A single convert execution is akin to writing and running an entire program in the Potter's Wheel language [46].

PALIMPZEST has a number of basic built-in Schemas to reflect common data types, such as File, TextFile, ImageFile, PDFFile, and so on. In some cases, we use built-in rules to perform convert operations among them. But most of the time we expect Schemas to reflect domain-specific user concerns and the system must dynamically figure out how to convert from one to another.

In some cases, the user's program may be ambiguous enough that several different convert implementations are arguably correct. This brings us to a core issue when designing PALIMPZEST: how to reason about quality and correctness.

3.3 Correctness and Quality

The biggest design challenges in PALIMPZEST lie in specifying correctness and managing quality.

Our design goal is to allow the user to treat data quality like she does performance in a modern RDBMS: It is usually good enough with no effort at all; but in rare cases where the quality is not good enough, improving through extra effort is possible. This philosophy extends to both the specification of correctness goals in the program and the practical achievement of high-quality data outputs at runtime.

Correctness Goals: During specification — that is, when the user is writing the program — there is ideally enough information in the source and destination schemas to fully indicate the desired AI steps. But what if the `ScientificPaper` Schema in Figure 4a had the field `institution`. Should the system populate it with the institution of the first author, or the most-frequently-seen institution among the others, or the institution associated with the publisher?

In most cases we expect a casual description of the goal, via field names and a simple text description, will be enough. But the user should be able to provide tighter specs, if desired. We are working on support for the user to provide validation examples. In all cases, constraints on correctness should be contained in the program text.

Output Quality: A related concern is improving the system's ability to deliver high-quality outputs once the user's specification is clear. There are two subproblems.

The first subproblem lies in finding approaches that maximize data quality, such as rewriting the prompt or marshaling multi-shot examples. To achieve this, we will rely on a mixture of strictly internal approaches (perhaps using DSPy-like prompt optimization methods [26]) and on allowing a "quality administrator" to provide out-of-band hints to the system, much like a traditional database administrator can create indexes that accelerate a user's relational query. In the limit, this approaches the challenge of creating a machine learning component from scratch. This will be much

```python
import palimpzest as pz

class ScientificPaper(pz.PDFFile):
  """Represents a scientific research paper"""
  title = pz.Field(desc="Title of the paper")
  authors = pz.Field(desc="List of author names")
  abstract = pz.Field(desc="The paper abstract")
  citation = pz.Field(desc="Paper citation")

# load pdfs, convert, and filter
pdfs = pz.Dataset(source="papers")
papers = pdfs.convert(schema=ScientificPaper)
papers = papers.filter(lambda p: p is not None)

##########################################
class DogBreed(pz.ImageFile):
  breed = pz.Field(desc="The breed of the dog")

# load images, convert, and filter
images = pz.Dataset(source="my-dog-pics")
breeds = images.convert(schema=DogBreed)
breeds = breeds.filter(lambda b: b is not None)
```

(a) Example showing two AI programs. In the first, a dataset of PDF files is converted into `ScientificPaper`. In the second, an image dataset is converted into `DogBreeds` if the image contains a dog.

operator	description
Project	π(rel., cols)
Select	σ(rel., predicate)
Convert	χ(rel., schema_a, schema_b)
Group By	Γ(rel., group_cond., agg.)
Limit	L(rel., limit)
Agg.	α(rel., agg_func)

(b) PALIMPZEST's full relational algebra. We extend the traditional relational algebra to include operators such as groupby which produce multiple relations.

Figure 4: PALIMPZEST code examples and a summary of its full relational algebra.

more challenging than improving traditional runtime performance, and will require engineers to become somewhat comfortable with soft quality guarantees. However, we would argue that years of experience with search and increased experience with neural models have already trained engineers to build systems that are robust to small quality issues. Moreover, the out-of-the-box experience with neural models keeps improving: there is an undeniable quality technology curve for PALIMPZEST to ride.

The second subproblem lies in accurately calibrating the quality of a novel plan. Many optimizations involve trade-offs against output quality, so measuring the possible impact is crucial. We can choose among many methods, ranging from the unsupervised, to the semi-supervised, to the fully-supervised. On the unsupervised side of the spectrum, we could simply choose an expensive "champion" model and treat its output as ground truth; PALIMPZEST can sample a candidate optimization's output and check to see if it matches the champion's. With a very small amount of coding effort by the quality administrator, we could also use distant supervision methods for detecting errors [47]. We expect full human supervision to only be available for the most important subtasks, but if it is, we have many options, such as fine-tuning task-specific models via reinforcement learning from human feedback (RLHF) [41, 45]. Someday, it may be possible to extend PALIMPZEST to be more fully integrated with end-user applications of the data it produces, and thereby marshal direct quality labels from end-user activity.

3.4 Cost Optimization Framework

At its core, PALIMPZEST allows users to define and execute *plans*, which are sequences of relational operations on datasets. By design, the declarative nature of these plans leaves many details of *how* to execute them underspecified. The key role of PALIMPZEST's cost optimizer is to identify physical implementations of these plans which are (near) optimal and align with user-specified preferences. The system diagram showing the path from program implementation, to generating and selecting the most cost-effective plan, and finally to executing that plan is shown in Figure 1. We reference steps in the diagram by number (e.g. step ①).

Developers first write declarative programs, such as the one shown in Figure 3. In this program, the user has specified a chain of Datasets and processing steps, which culminate in the final `emails` set. Upon calling `Execute()` on line 17, that chain is sent to the **Program Optimizer** for compilation into an initial logical plan (step ①). The set of logical operators is shown in Figure 4b. Given this initial logical plan, PALIMPZEST performs logical optimization to create a set of new, functionally equivalent plans which may have different cost, runtime, and quality trade-offs (step ②). Many of these optimizations reuse traditional ideas from query optimization in databases, such as predicate pushdown, projection pushdown, and filter reordering. However, implementing these optimizations in PALIMPZEST presents new challenges. For example, effectively implementing projection pushdown may require determining the best way to split a single convert operation into multiple operations.

The resulting logical plans are then used to generate an even larger set of candidate physical plans (step ③). This is akin to the physical optimization stage in relational databases. At this stage, the system hypothesizes a large number of

concrete plans that are consistent with the input logical plans, but which make a range of detailed decisions specific to AI systems. These include decisions around model choice, k-v cache management, prompt generation, token reduction, and other areas. One LLM-specific optimization we implement is to combine multiple subtasks into a single LLM query (such as asking for both the `title` and `authors` of a scientific paper) in order to avoid unnecessary token duplication across tasks. This is similar to FrugalGPT's *query concatenation* method [11], but works at the compiled subtask level.

Note that much of the flexibility of the PALIMPZEST optimization process comes from the wide number of possible implementations for even one particular convert operation. For example, converting a `TextFile` to an `Email` could be done by formulating an LLM task or by synthesizing appropriate traditional code. Converting a `PDFFile` to a `ScientificPaper` could be done by using a set of LLM tasks or perhaps by training a conventional local NLP model. LLM tasks can be modified by choosing a different model, by changing the prompt, by combining multiple tasks into a single prompt, or in some cases even by first fine-tuning a model. Each of these options presents different trade-offs. Of course, in some cases users may wish to explicitly state how to perform some operations, in which case they are free to provide UDFs — but this is not required.

At this point, the Program Optimizer has created a potentially large set of programs that are consistent with the user's input program and which operate at different points in the optimization space of runtime, financial cost, and quality. However, PALIMPZEST still needs to compute estimates of these metrics for each physical plan (steps ④ and ⑤). To accomplish this, the **Plan Executor** executes a small set of **sentinel plans** to gather sample data on plan execution statistics. We provide more details on this estimation procedure in Subsection 4.3. The Program Optimizer then chooses the best physical plan, according to per-plan estimates and the user's preference (step ⑥). Finally, this choice is sent back to the Plan Executor, which executes the plan and spends computation and financial resources, possibly drawing on many external model and data service providers (step ⑦).

3.5 Dataset Registration and Result Caching

Users must preregister named base-level source `Datasets` — like `enron-emails` in Figure 3 — before they can be consumed by PALIMPZEST operators at runtime. Giving every dataset an unambiguous name is meant to eventually allow an AI program to be run in different locations on the exact same set and thereby yield identical results. (However, the current prototype supports only local names. We will support a global dataset naming service in the future.) This registration scheme also eliminates conventional out-of-band data marshaling errors as a potential source of bugs.

The PALIMPZEST standard library implements a number of common datasource types already, including bytes from a single file or a single directory. In the future we will support relational databases, AWS S3, and other data sources. Users can also easily write custom datasources for their specific use cases.

Given the low-throughput associated with invoking LLM models and services, PALIMPZEST makes a best-effort attempt to avoid sending requests to an LLM unless it is absolutely necessary. As a result, caching is a key component of the PALIMPZEST system. The system currently caches intermediate results at the granularity of a `Dataset`, but in the future we may modify this to cache individual records. Thanks to unambiguous base-level `Dataset` names, PALIMPZEST can detect when two different programs share computation, and thus when a cached intermediate result from program A can be re-used by program B.

The caveat with caching is that different implementation decisions can yield results of different quality, so two runs of the same program may yield different results. In the future, the caching layer will record quality statistics with each cached intermediate result, so the system can invalidate previously-cached results when user quality preferences change.

4 Program Optimization

Managing and exploiting a large space of useful optimizations is PALIMPZEST's core feature. In this section we describe key logical and physical optimizations that can be used by PALIMPZEST — many of which have already been implemented in our prototype.

4.1 Logical Optimizations

PALIMPZEST has a set of logical optimizations which can be applied to the logical plan implied by a user's program. These optimizations create new plans that are logically equivalent, but may yield significant runtime and cost savings depending on factors like operation execution cost and the selectivity of filter operations. We have implemented two logical optimizations already — filter reordering and convert reordering — but we plan on adding more in the future (such as splitting and coalescing LLM requests).

Filter Reordering simply permutes the ordering of selection filters in a logical plan wherever possible. For example, if an input logical plan has the filter sequence A, B, and C, this logical optimization would yield 5 additional plans, one for each unique permutation of filters. If each of the filters has a unique selectivity, then exactly one of these permutations will minimize the number of records processed by the system. PALIMPZEST does not initially know these selectivities (though future versions may use historical data). However, PALIMPZEST's cost optimizer is capable of estimating these selectivities after execution begins and sample execution data is collected. Thus, by considering all filter reorderings in its logical optimization stage, PALIMPZEST can find more efficient executions of user plans.

Convert Reordering enables PALIMPZEST to move convert operations around the logical plan, similar to filter reordering. This can be especially beneficial when a plan has a mixture of convert operations and filter operations, because moving an expensive convert operation after a filter which does not depend on its output can save unnecessary computation. For example, in the Real Estate Search workload, the fastest plan will apply the highly-selective text-based filters before it runs the expensive image processing step for testing whether a listing is "modern and attractive." At the moment, PALIMPZEST relies on the programmer to explicitly state dependencies between convert and filter operators. In the future, we hope add the capability for PALIMPZEST to deduce this information automatically.

4.2 Physical Optimizations

Some recent AI optimizations, such reducing text generation latency via speculative inference [36, 50, 29, 10], are good ideas that can be deployed in many contexts, including chat, general LLM API services, and PALIMPZEST. But some optimizations are better or are only possible because our system's declarative programming framework permits extreme flexibility around implementing the user's goal, and thus are unavailable to lower-level components of the AI stack.

Model Selection is maybe the simplest. A high-level AI program comprises many tasks; for example, the code in Figure 3 entails extraction of the `Email` fields from unstructured text as well as filtering the emails by content. Thanks to its declarative language, PALIMPZEST can decompose the high-level program into smaller tasks and choose the most appropriate model for each. It might be fine to use a cheap, small, fast model for easy tasks, and only use the expensive model for harder ones. The idea is simple, but implementing it is not: because (1) a single program can comprise many tasks, (2) the exact task decomposition can change depending on other optimizations, (3) the difficulty of a task can change over time, and (4) model quality can fluctuate as LLM services make updates. This optimization can easily become burdensome without PALIMPZEST's help.

Code Synthesis involves generating synthesized code to perform specific tasks dynamically. In certain real-world scenarios, tasks may not require deep semantic understanding and can be efficiently handled through synthesized code. By replacing calls to LLMs with calls to synthesized functions, we can generally reduce runtime and lower costs. (The EVAPORATE system [5] used this approach, along with some weak supervision methods [47], to replace some LLM operations.) PALIMPZEST analyzes a set of sample inputs for a given conversion operation and then employs an LLM to generate a function that performs the necessary conversion. This approach can streamline processes where semantic depth is unnecessary, optimizing both performance and resource utilization.

Input Token Reduction aims to delete portions of certain tasks' input data while still obtaining high-quality results. For example, it should be possible to populate the `title` and `authors` fields of `ScientificPaper` in Figure 4a while ignoring almost all of the input text. In document-processing use cases, this reduction in token size — and thus reduction in financial cost and increase in execution speed — can be very dramatic. By observing multiple naive examples of using `convert()` to transform a PDF to a `ScientificPaper`, the declarative optimizer can run automatic "experiments" to determine which regions of the input are necessary. In some ways, this optimization is akin to a "micro-RAG" task, choosing salient excerpts from a source object. However, unlike a full RAG system, this optimization exists only for the life of the program execution. This is possible because PALIMPZEST can learn task properties at the schema level. In the naive chat-processing use case, which lacks schemas and a visible workload of tasks from a single program, it is not clear how this method could be implemented.

Multi-data Prompt Marshaling is the problem of deciding how user tasks should be decomposed into prompts. Should data objects be processed in a row- or column-centric manner? For example, when running the code at the top of Figure 4a, computing all of the `ScientificPaper` fields in a single LLM call might allow us to process the input tokens just once, while obtaining multiple output values. But if there is a filter on the `title` field, and if the `citation` output field is very large, then a "column"-centric approach might be better. Since the best decision will depend on the current state of the AI program and its input data, a human engineer attempting to implement this optimization manually would need to constantly reevaluate and possibly reimplement it.

Output Token Reduction aims to reduce the size of LLM generation outputs without changing the application-specific quality of these outputs. SplitWise [43] showed that LLM inference time is generally proportional to the size of the input and output tokens, with the output tokens having a greater impact when various LLM optimizations are activated.

When PALIMPZEST is asked to find large excerpts from input objects, such as populating the `citations` field of `ScientificPaper`, the output token size can be very large. In these cases, it may be possible to reformulate the naive task so it gives the same answer but with dramatically smaller output. For example, the system can insert artificial tokens into the input text and then ask the LLM to use these tokens to report which text block contains the target result. In some early test cases, this approach can reduce output sizes from thousands of tokens down to just two.

Model Cascades is a method that many researchers have explored in the context of image processing [25, 4, 8, 51, 6]. The core idea is to construct and exploit a series of models that accomplish the same goal, ranging from fast/low-quality models on the low-end to expensive/high-quality models on the high-end. Task processing starts on the inexpensive model. If the model can process the input with high confidence, the system uses its output; if not, the system gives the input to a more expensive model in the sequence. This approach is not implemented in PALIMPZEST yet, but it has been very successful in other related projects.

Knowledge Distillation methods approximate the outputs of an expensive large-parameter model by using a smaller, cheaper (and often more limited) replica model [20, 22, 19]. Methods such as the *teacher / student* approach have produced small models that perform similarly to much larger ones, such as Alpaca and Vicuna [52, 12]. In certain cases, it may be feasible for our system to perform per-task knowledge distillation to obtain a small and fast model.

Workload-Aware Execution Management, enabled by PALIMPZEST's ability to design entire workloads of model requests, should yield opportunities in model serving and resource management. If a distributed model service permits clients to give batch-oriented optimization hints — such as whether a set of queries will use the same model, or to implement prefill prepacking [63] — PALIMPZEST can exploit it. Strategically co-scheduling LLM tasks with similar prompts can optimize key-value (KV) cache reuse, substantially improving the KV cache hit rate during LLM inference [35, 60]. Because the system can reformulate prompts and gather statistics about output token lengths, it should be able to design batches of model requests based on similar output token lengths, thereby minimizing waiting times for requests that finish early and boosting task throughput.

4.3 Choosing An Optimization

In order to be valuable, the system must not just hypothesize a valuable set of optimized plans, it must actually choose one that yields concrete benefits. PALIMPZEST follows steps (marked with circled numbers) described visually at the top of Figure 1, and algorithmically in Algorithm 1.

Algorithm 1 Optimized Plan Selection Algorithm

Require: $userCode$, $userPolicy$ # Step ①
1: $logicalPlans$ = generateLogicalCandidates($userCode$) # Step ②
2: $sentinelPhysicalPlans$ = getPhysicalPlans($logicalPlans$, $sentinel = True$) # Step ③
3:
4: $performanceStatisics$ = { }
5: **for** 0...NUM_SAMPLES **do**
6: $input$ = getSampledInput()
7: $stats$ = runAndComputeStatistics($sentinelPhysicalPlans$, $input$) # Step ④
8: performanceStatistics.update($stats$) # Step ⑤
9: **end for**
10:
11: $physicalCandidates$ = getPhysicalPlans($logicalPlans$, $stats = performanceStatistics$)
12: $reducedCandidates$ = naiveElimination($physicalCandidates$)
13: $frontierCandidates$ = scoreAndEliminatePlans($reducedCandidates$, $performanceStatistics$)
14:
15: **return** chooseBestPlan($frontierCandidates$, $userPolicy$) # Step ⑥

The algorithm starts with a user program and an optimization goal (such as "minimize runtime subject to F1 > 0.5"). On line 1, it generates all possible logical plans consistent with the user program, as described in Subsection 4.1. On line 2, it generates a small number of "sentinel" physical plans that are consistent with these logical plans. A sentinel plan is a simple hard-coded plan that should shed information on different selectivities and the accuracy of individual non-optimized operators. We currently generate three sentinel plans: one that uses GPT-3.5 for each convert and filter, one that uses MIXTRAL 8x7B, and one that uses GPT-4. (These are depicted in the sample-based statistics collection module at the top-right of Figure 1.)

On lines 4-8, the algorithm runs all of the sentinel plans on a small NUM_SAMPLES number of inputs. The goal is to collect basic statistics about the behavior of some easy-to-understand plans.

On line 10, the system hypothesizes physical plans that embody optimizations for the logical plans. This set can be quite large. Our prototype generates 234, 10,140, and 1,950 physical plans for the Legal Discovery, Real Estate, and Medical Schema Matching programs, respectively. On line 11, we use a small set of rules to eliminate plans that likely do not add to the set of useful runtime options. On line 12, the algorithm uses statistics from lines 4-8 to score the expected runtime, financial cost, and quality of all the remaining physical plans; it then throws away any plans that are not on the Pareto frontier.

Finally, on line 14, the algorithm scores each frontier candidate in light of the user's optimization goal and returns the highest-scoring option for execution. Estimating runtime is straightforward using well-known relational database planner methods, while estimating cost simply requires combining published service cost data with an accurate token consumption estimate. Measuring quality is much harder, as described in Subsection 3.3. The current system simply uses the champion model approach.

The algorithm is admittedly naive. For example, it does not have a rigorous termination criterion for the sampling phase on lines 5-8, and the sentinel plans are chosen arbitrarily. But as we will see in Section 5, the algorithm effectively identifies plans that are close to the true optimal plan within the set generated by PALIMPZEST.

5 Evaluation

We evaluated PALIMPZEST on the three workloads motivated in Section 1. We first describe our current prototype, a PALIMPZEST program for each workload, and our evaluation methodology. We then examine two experimental claims. First, PALIMPZEST can use its three implemented optimizations to obtain physical plans that offer diverse performance trade-offs, some of which are more appealing than unoptimized naive plans. Second, the PALIMPZEST optimizer can predict plan properties quickly and accurately enough to choose a plan that fits user preferences better than unoptimized naive plans. Finally, we showcase PALIMPZEST's ability to leverage system parallelism to minimize workload runtimes.

5.1 Our Prototype

We have implemented an early PALIMPZEST prototype in about 9,200 lines of Python code, which we have made available at `https://github.com/mitdbg/palimpzest`. It implements the operators in Figure 4b, and can run many programs, including the three test workloads described in Section 1 — Legal Discovery, Real Estate Search, and Medical Schema Matching.

We have implemented three of the optimizations described in Section 4: model selection, code synthesis, and input token reduction. We currently test the system using the `gpt-3.5-turbo-0125`, `gpt-4-0125-preview`, and `gpt-4-vision-preview` OpenAI models [40] and the `Mixtral-8x7B-Instruct-v0.1` model served by the Together.ai API [3]. We also use the Modal online service [1] for bulk non-AI function execution, such as parallel PDF processing and equation image extraction and conversion. We have tested the system with local model execution (via the Ollama [2] framework), but so far we have rarely found this to be an appealing option.

PALIMPZEST is implemented using the iterator model. Thus, plan execution proceeds one record at a time with each operator blocking until it receives the necessary input record(s) from its source operator(s). For clarity's sake, most of our experiments report simple single-threaded execution time, so we can better show the work saved by system optimizations. However, many operations — including the convert operator — have parallel implementations that take advantage of parallelism offered by the underlying service or hardware, and we report a few parallel numbers.

5.2 Evaluation Workloads

We evaluated the model selection, code synthesis, and input token reduction optimizations described in Section 4 using the three workloads first discussed in Section 1. Below, we provide details on the implementation and data used for each workload.

Legal Discovery. We implemented this task using the source code shown in Figure 3. The program loads the raw text files and stores their `filename` and `contents` in a `pz.TextFile`. On line 9, the program converts the input data to the `Email` schema, computing the `sender` and `subject` fields. Lines 10 and 11 filter the email by content.

We created a test dataset of 1000 emails drawn from the Enron email collection [14]. This dataset was curated and labeled by hand to contain 50 examples that discuss the management of fraudulent investment vehicles, with another 30 that contain fraud-related text but are not fraudulent in nature. The remainder were randomly chosen. We registered this

```python
import palimpzest as pz

class RealEstateListingFiles(pz.Schema):
    """The source text and image data for a real estate listing."""
    listing = pz.StringField(desc="The name of the listing")
    text_content = pz.StringField(desc="The content of the listing's text description")
    image_contents = pz.ListField(element_type=pz.BytesField, desc="A list of the image contents")

class TextRealEstateListing(RealEstateListingFiles):
    """Represents a real estate listing with specific fields extracted from its text."""
    address = pz.StringField(desc="The address of the property")
    price = pz.NumericField(desc="The listed price of the property")

class ImageRealEstateListing(RealEstateListingFiles):
    """Represents a real estate listing with specific fields extracted from its images."""
    is_modern_and_attractive = pz.BooleanField(desc="The home interior is modern and attractive")
    has_natural_sunlight = pz.BooleanField(desc="The home interior has lots of natural sunlight")

def within_two_miles_of_mit(record):
    # filter based on record.address

def in_price_range(record):
    # filter based on record.price

# define logical plan
listings = pz.Dataset(source="real-estate-eval", schema=RealEstateListingFiles)
listings = listings.convert(TextRealEstateListing, depends_on="text_content")
listings = listings.convert(ImageRealEstateListing, depends_on="image_contents")
listings = listings.filter(
    "The interior is modern and attractive, and has lots of natural sunlight",
    depends_on=["is_modern_and_attractive", "has_natural_sunlight"]
)
listings = listings.filter(within_two_miles_of_mit, depends_on="address")
listings = listings.filter(in_price_range, depends_on="price")

# create and execute physical plans...
```

Figure 5: The AI program written using PALIMPZEST for the Real Estate Search workload.

set of 1000 emails using the identifier "enron-eval". The goal of the program is to recover only those emails that indicate genuine fraudulent activity. For all the experiments on this workload, we evaluated the F1-score of the program's output using the manual labels described above. (Of course, during optimization and plan selection, PALIMPZEST did not have access to these labels and had to estimate the plans' quality in an unsupervised fashion.)

Real Estate Search. Our PALIMPZEST program for this workload is shown in Figure 5. It has the same rough form as the Legal Discovery task above, with a few key differences. First, because the real estate listings are multimodal, they currently do not fit into a single core library schema such as `pz.TextFile`. Thus, we first define a few utility schemas. `RealEstateListingFiles` stores the raw image and text data for each listing; the data loading is performed by a simple user-defined datasource that we have omitted for brevity. We also define `TextRealEstateListing` and `ImageRealEstateListing` for the text and image attributes that we wish to compute [2]. Second, on lines 33 and 34 the user provides user-defined functions for selection, instead of text strings. Finally, several operations on lines 27-34 use the `depends_on` parameter (which was described in Subsection 4.1), in this case, allowing the system to skip image processing entirely.

We manually scraped 100 real estate listings from Boston and Cambridge, MA to obtain their natural language text descriptions and the first three deduplicated images from each listing. We registered this dataset using the identifier "real-estate-eval". For each listing we manually labeled whether it satisfied the location, price, and "attractive and modern" and "natural sunlight" criteria, allowing us to compute an F1-score for any program output. Overall, 23 of the 100 data items satisfied all criteria.

Medical Schema Matching. The PALIMPZEST program for this task is shown in Figure 10. This program aims at reproducing the output of the data harmonization pipeline performed by the authors of [31]. The authors provide a single table produced as the result of collecting data from 11 different experimental studies. All tables from the different studies have been manually integrated to provide a unified view of patient information for tumor cases analyzed in the source studies. This is a complex and time-consuming task, even for domain experts. According to the author contribution section, 66 out of 79 paper authors focused on the data curation task. However, with PALIMPZEST, we can implement the pipeline in approximately 30 lines of code. Lines 3-21 of the program reflect a detailed description of the target output schema, as described in the paper. This program is similar to the programs for Legal Discovery and

[2] At the moment, writing two schemas is necessary because schemas and convert operations have a one-to-one relationship in our system, but in the future we plan to support splitting a single schema conversion across multiple convert operations.

Real Estate Search, but its unique feature is the use of the `cardinality` parameter, which allows PALIMPZEST to yield multiple output objects from a single input object, i.e., multiple tables from a single spreadsheet, and multiple case data from the tables.

We curated a dataset of 11 spreadsheet files — containing a total of 49 tables — from the supplemental data provided in the original works [9, 13, 16, 24, 30, 18, 27, 39, 48, 54, 55]. In Figure 10, we refer to this dataset as "medical-eval". The first goal of the program is to filter, out of the 49 input tables, only those which contains information about patient case data. Once these tables are identified, the program aims at mapping each column in the source tables to a target column in a set of 15 attributes. This task is very challenging: not all target attributes can be found in all source tables, and neither there is guarantee that matching columns in the source tables have the same representation as the target columns.

For this workload, we measure accuracy using manually annotated ground truth data matching the columns from the source tables to those of the harmonized table provided by the authors of [31]. We report the micro-average of the F1-score across all target attributes and paper studies.

5.3 Optimizations Produce Plans with Diverse Performance Trade-offs

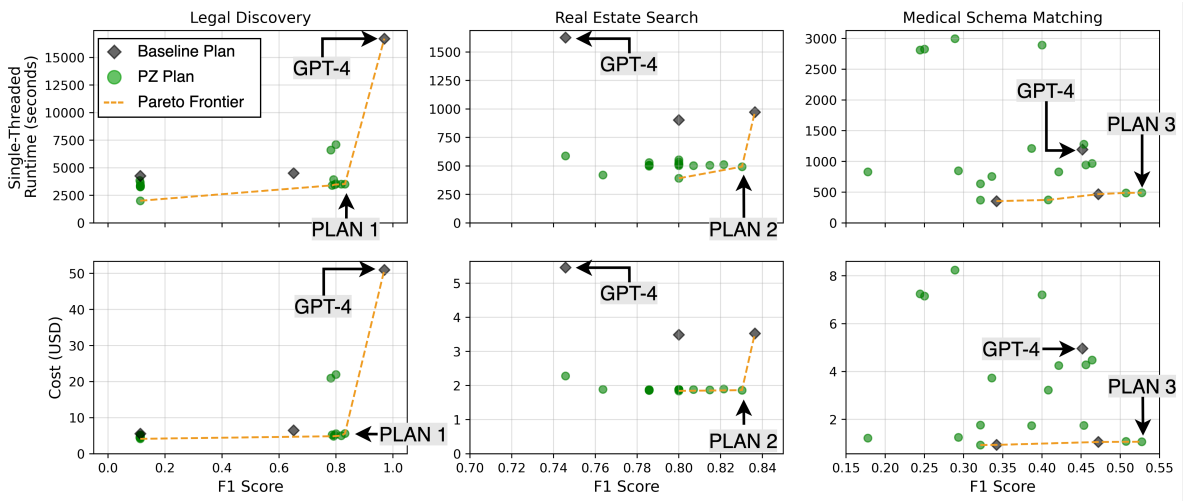


Figure 6: Performance of different plans on each workload (plans towards the bottom-right of each subplot are better). Naive plans are depicted with black diamonds and plans which PALIMPZEST creates with its physical and logical optimizations are shown with green circles. Across all workloads, PALIMPZEST produces plans on the Pareto frontiers of runtime vs. quality and cost vs. quality.

Our first experimental claim is that PALIMPZEST can use its three optimization strategies to create a set of physical plans that offer appealing trade-offs regardless of the user policy. To evaluate this claim, for each workload we ran PALIMPZEST up to (but not including) the final step in Algorithm 1. We then took the set of frontier plans, three baseline plans, and the top-k plans from the *reducedCandidates* that were closest to the approximated Pareto frontier, such that we ultimately executed 20 plans in total. Our three baselines were the naive physical plans for each workload which used only GPT-4, GPT-3.5, or Mixtral-8x7B, respectively.

Figure 6 shows the observed runtime, cost, and quality that came from executing all the aforementioned plans. Some plans exhibit similar performance, which makes them overlap in the figure, thereby presenting fewer than 20 distinct visual data points. Plans closer to the bottom-right of each subplot are better. Note that data points reflect observed values, not the system's estimated ones. During sample-based statistics collection, we used 5% of the workload's total input size to run our sentinel plans, which gathered data to help estimate the cost of all plans. The one exception to this was the Medical Schema Matching workload, where we used 1 out of 11 inputs to gather sample data.

We found that PALIMPZEST is able to hypothesize useful plans at a number of different points in the trade-off space. On the Legal Discovery workload, PALIMPZEST was able to suggest physical plans (e.g. PLAN 1) that dominate the GPT-3.5 and Mixtral baselines, with an F1-score that is 7.3x and 1.3x better (respectively) at lower runtime and cost. Compared to the GPT-4 baseline, PALIMPZEST produced a cluster of plans near PLAN 1 that are $\sim$4.7x faster and $\sim$9.1x cheaper, while still achieving up to 85.7% of the GPT-4 plan's F1-score.

We saw similar results on the Real Estate Search workload, where PALIMPZEST once again generated physical plans which offer desirable trade-offs relative to baseline plans. PALIMPZEST was able to produce physical plans (e.g. PLAN 2) that obtained 3.3x lower runtime, 2.9x lower cost, and up to 1.1x better F1-score than the GPT-4 baseline. These performance improvements are especially impressive considering the majority of the cost and runtime on this workload are dominated by calls to the vision model — which cannot be optimized away using the methods in our current prototype. (In the future, we will explore options for visual processing optimizations.)

Finally, on Medical Schema Matching — our most challenging workload — PALIMPZEST was able to produce plans (e.g. PLAN 3) which have $\sim$ 2.4x lower runtime, $\sim$ 4.6x lower cost, and up to 1.2x better F1-score than the GPT-4 baseline. This workload is especially challenging for PALIMPZEST because the code synthesis and token reduction optimizations are not very effective, but the system still identified plans that completely dominate the performance of a GPT-4 baseline.

We have shown that PALIMPZEST can generate plans like PLAN 1, 2, and 3 (plan details are shown in the Appendix), which provide users with compelling performance trade-offs. However, this does not automatically confirm that PALIMPZEST's optimizer will choose these plans during runtime. In the following section, we demonstrate that for a variety of policies, PALIMPZEST's cost optimizer does indeed identify and select such plans.

5.4 Cost Optimizer Selects Plans with Significant Performance Improvements

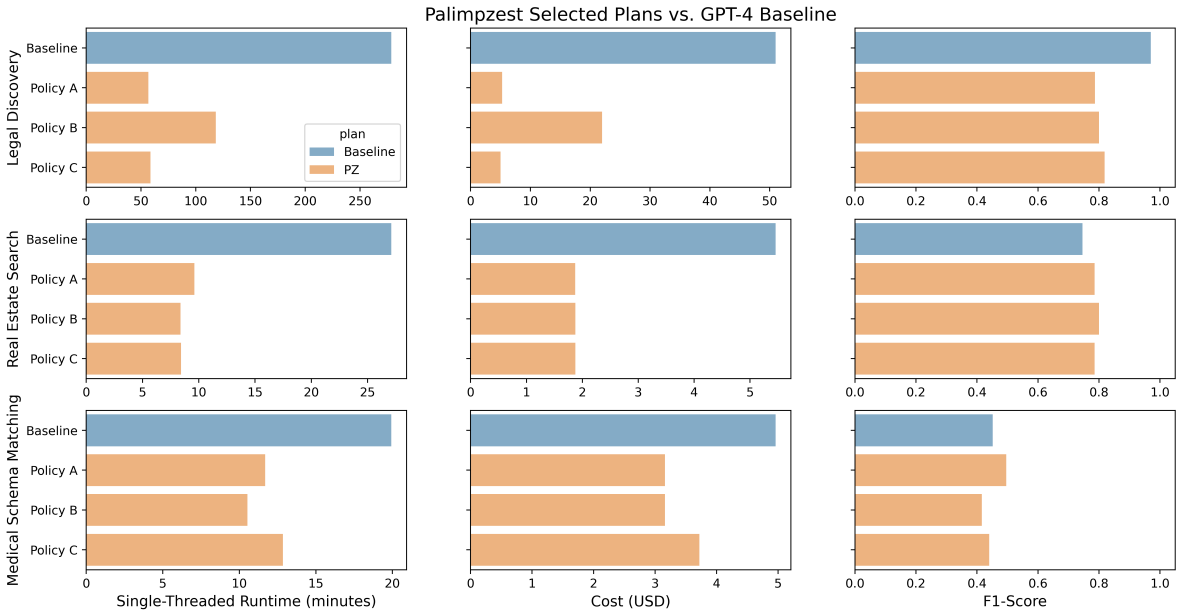


Figure 7: Comparing the performance of plan(s) selected by PALIMPZEST to the baseline GPT-4 plan for three different policies on each workload. **Policy A** maximized F1-score at cost $<$ \$20, \$3, and \$2 for each workload (top-to-bottom); **Policy B** maximized F1-score at runtime $<$ 167m, 10m, and 16.7m for each workload (top-to-bottom); and **Policy C** minimized cost at F1-score $>$ 0.8, 0.8, and 0.4 for each workload (top-to-bottom). PALIMPZEST consistently finds plans which provide similar F1-scores and lower costs and runtimes than the GPT-4 baseline.

Our second experimental claim is that PALIMPZEST can obtain executable programs that have better end-to-end runtime, cost, and quality than a conventional naive program. To evaluate this claim, we ran the system as described in Algorithm 1 — this time including the final step which chooses the best plan for a given policy. We ran three policies for each workload. **Policy A** was to maximize quality at cost $<$ \$20.0, \$3.0, and \$2.0 for Legal Discovery, Real Estate Search, and Medical Schema Matching, respectively. **Policy B** was to maximize quality at runtime $<$ 10,000s, 600s, and 1000s (for the same order of workloads). Finally, **Policy C** was to minimize cost at an F1-score $>$ 0.8, 0.8, and 0.4 (for the same order of workloads). These fixed cost, quality, and runtime thresholds were set to be challenging, yet physically attainable based on our results in Subsection 5.3. (The full set of thresholds can be found in Table 1.)

Figure 7 presents our results across three performance metrics, with each metric displayed in a separate column. The results are organized by workload, with each workload represented in a row of subplots. Within each subplot, results are

Table 1: PALIMPZEST satisfied the policy constraint on 7 out of 9 plans it executed in Subsection 5.4. Constraint thresholds were chosen to be challenging, but realistic based on our results in Subsection 5.3.

Policy	Workload	Actual v. Constraint
Max F1 @ Cost $<$ \$20	Legal Discovery	**\$5.27$<$\$20**
Max F1 @ Cost $<$ \$3	Real Estate Search	**\$1.87$<$\$3**
Max F1 @ Cost $<$ \$2	Medical Schema Matching	\$3.16$\not<$\$2
Max F1 @ Runtime $<$ 10000s	Legal Discovery	**7102s$<$10000s**
Max F1 @ Runtime $<$ 600s	Real Estate Search	**502s$<$600s**
Max F1 @ Runtime $<$ 1000s	Medical Schema Matching	**632s$<$1000s**
Min Cost @ F1 $>$ 0.80	Legal Discovery	**0.82$>$0.80**
Min Cost @ F1 $>$ 0.80	Real Estate Search	0.79$\not>$0.80
Min Cost @ F1 $>$ 0.40	Medical Schema Matching	**0.44$>$0.40**

further divided by the physical plan selected by the optimizer, with each plan occupying a separate row. We compared the plans chosen by PALIMPZEST to a baseline plan, which employs GPT-4 for all conversion and filtering operations.

Overall, we found that PALIMPZEST identifies plans in the space of physical candidates which (1) offer significant performance improvements over the GPT-4 baseline, (2) generally satisfy policy constraints (7 out of 9 satisfied), and (3) have speedups and cost savings which outweigh the overhead of collecting sample data. For example, on the Legal Discovery workload (first row in Figure 7), the plan selected by PALIMPZEST for **Policy A** achieved a runtime and financial cost that are 80.0% and 89.7% lower than the GPT-4 baseline, respectively, at an F1-score within 81.1% of the baseline. The system achieved similar results for **Policy C**, with slightly better quality (within 84.3% of the baseline).

For the Real Estate Search workload (second row in Figure 7), the plans chosen by PALIMPZEST achieved (on average) 67.5% lower runtime, 65.7% lower cost, and 6% better F1-score than the baseline GPT-4 plan. Finally, on the Medical Schema Matching workload (third row in Figure 7), PALIMPZEST was able to identify plans that provide up to 47.2% and 36.3% lower runtime and cost, respectively, at comparable F1-scores to the GPT-4 baseline.

5.5 Minimizing Runtime with Parallel Operators

Table 2: Runtime speedup from executing PALIMPZEST plans with parallel convert and filter operators. We compare these to the single-threaded GPT-4 baselines for each workload. All PALIMPZEST plans were selected by the system under **Policy A**.

(a) Legal Discovery

Plan	Runtime (s)	Cost (\$)	F1
Single-Threaded Baseline (GPT-4)	16,712	51.0	0.97
Palimpzest	185 (1.1%)	5.60 (11.0%)	0.81 (83.5%)

(b) Real Estate Search

Plan	Runtime (s)	Cost (\$)	F1
Single-Threaded Baseline (GPT-4)	1,626	5.46	0.75
Palimpzest	80.9 (5.0%)	1.86 (34.1%)	0.80 (107%)

(c) Medical Schema Matching

Plan	Runtime (s)	Cost (\$)	F1
Single-Threaded Baseline (GPT-4)	1,195	4.96	0.45
Palimpzest	215 (18.0%)	3.36 (67.7%)	0.46 (102%)

For our final evaluation, we ran PALIMPZEST with parallel implementations of the convert and filter operations to demonstrate the system's ability to achieve large runtime speedups — with competitive costs and F1-scores — relative to a single-threaded baseline plan. For each convert and filter operator we used 32 workers to execute operations in parallel. We did not pipeline operations, i.e. every record needed to be processed in a given convert or filter operation

before the next operation in the plan could commence (but this is an optimization we could implement in the future). For each workload we ran PALIMPZEST and asked the optimizer to select the optimal plan according to **Policy A**.

The results of our evaluation are shown in Table 2. We can see that on Legal Discovery, PALIMPZEST achieved a **90.3x speedup at 9.1x lower cost while obtaining an F1-score within 83.5% of the GPT-4 baseline**. On Real Estate Search and Medical Schema matching, the optimized plan dominated the GPT-4 baseline on all metrics. **These plans achieve better F1-scores than their baselines, and do so with speedups of 20.0x and 5.6x as well as cost savings of 2.9x and 1.5x, respectively.**

We do not use any exotic algorithms, and of course it is straightforward to run model prompts in parallel. PALIMPZEST's abstractions simply allow the system to obtain these speedups with no additional work by the user.

6 Related Work

There is a large and growing literature in programming frameworks for LLMs, foundation models, and other AI artifacts.

Many current frameworks are focused on prompt management. LangChain [15] and LlamaIndex [34] are popular libraries for building LLM applications, with special support for mechanical issues surrounding AI model use: managing prompt templates, handling user-provided labeled examples, managing document tokenization, and so on. They have no higher-level task representation, although there is support for standard use cases like chatbots and RAG.

DSPy [26] focuses on creating high-quality prompts. It takes a high-level description of a user's LLM goal and yields a set of concrete prompts (or even LLM operations like fine-tuning) to implement the user's goal. Like PALIMPZEST, DSPy aims to abstract some details of the AI programming process. But DSPy's goals are different: it focuses entirely on improving prompt quality, ignores most performance and cost issues, and asks the programmer to make decisions about its machine learning algorithm. We have used DSPy in some cases when compiling individual tasks inside PALIMPZEST.

Several other projects — such as Outlines [56], Guidance [21], and RELM [28] — attempt to control how LLMs emit data. These projects are especially useful when producing data that is intended for machine consumption, such as producing a JSON record that conforms to a particular schema, or emitting classification labels drawn from a target vocabulary. Outlines and RELM offer runtime optimizations that avoid inference costs when the syntax fully determines the output. These are useful systems but are also fairly narrow and do not address our target workload.

SGLang [64] is an ambitious project that combines a number of features present in the above systems, such as prompt templating methods and constrained outputs. It also tries to offer some performance features not present in the above systems, such as parallel execution and batching. It even offers management of multimodal data. However, the Structured Generation Language is still fairly low-level and asks developers to make many decisions manually, such as the direct text of a prompt, the desired elements that comprise a batch of prompts; or the degree of parallelism. It might make sense for a future version of PALIMPZEST to use SGLang as a compiled runtime description language.

The SkyPilot system [58] is similar to PALIMPZEST in its emphasis on cost reduction of ML workloads, but is primarily concerned with efficiently deploying coarse DAG of tasks across broadly-similar cloud platforms. It offers a broker that stands between a user's workload and advertised services from competing cloud services. It aims for an ideal assignment of work to resources, rather than formulating efficient program-specific implementations.

FrugalGPT [11] is a high-level library meant to be deployed on top of LLM access packages such as OpenAI [40]. It offers a number of valuable optimizations, some of which resemble the optimizations in Section 4. The FrugalGPT vision of appealing AI optimizations, combined with some trade-offs, is entirely consistent with PALIMPZEST's goal and approach, and the observed improvements in runtime and cost are strong. However, it is unclear how the user describes tasks to the FrugalGPT's optimizer, so the scope for future optimization is also unclear. Moreover, it is not clear whether FrugalGPT optimizations can be implemented in the context of a broader AI program that includes not just LLM processing but also non-LLM AI components and conventional data processing elements.

AutoGen [57] is an interesting development framework for "LLM applications," but with a domain focus that is dramatically different from the one we pursue. This system imagines LLM applications as conversations among agents, which might be humans, LLMs, or tools. The AutoGen developer aims to design the conversation and control flow that takes place among these agents. This vision of LLM applications is driven by a fundamentally different model from our own, which is based more in the data processing tradition.

Urban and Binnig proposed "Language-Model-Driven Query Planning" in the Caesura system [53]. The goal is to allow users to describe a data processing task in natural language, which yields an executable query plan, in particular one that can operate on multimodal data. This work is similar to PALIMPZEST in that it yields a query plan that combines

traditional relational operators with model-driven ones. However, PALIMPZEST makes no attempt at all to build the logical query plan from natural language; we expect a human programmer to provide the core processing logic. The published Caesura system does not perform any plan optimization, although the authors describe this as an area for future work.

ZenDB [33] is a document analytics system that enhances query performance by extracting semantic hierarchical structures from templatized documents and integrating a query engine that leverages these structures to efficiently and accurately execute SQL queries on document collections. ZenDB employs optimization techniques such as predicate reordering, pushdown, and projection pull-up to refine the execution of user-specified SQL queries. This optimization is facilitated through a summary-based tree search for each document, aiming to minimize cost and latency while maximizing accuracy. Although ZenDB and similar systems share optimization objectives with PALIMPZEST, their efforts primarily concentrate on logical optimization and are predominantly focused on managing text data. In contrast, PALIMPZEST utilizes multiple strategies to optimize broader aspects of query execution.

The Evaporate system proposed by Arora et al. in [5] shares some intuitions of PALIMPZEST regarding the trade-off of cost/quality in LLM workloads using code generation. However, while Evaporate is designed specifically for an information extraction use case and employs only static prompts and code generation, PALIMPZEST provides a more general approach to express a wider class of ML workloads, with several optimization strategies alongside code generation, e.g., model selection and token reduction. As such, we believe the more advanced weak-supervision code generation strategy proposed in [5] could be integrated as one of the optimizations within the PALIMPZEST framework.

Surprisingly, PALIMPZEST has some commonalities with CrowdDB [17], Qurk [38], and Deco [42]; systems that aim to provide a declarative approach to create data processing pipelines with crowd-sourcing operations to answer queries that cannot "be answered by machines only" [17]. For example, [17] automatically optimized data processing pipelines with crowd-sourcing operations for entity matching or (open) data retrieval, whereas [37] introduced and optimized crowd-sourcing operators to extract data from images. It turns out that — with the power of LLMs — some of the tasks no longer need humans and can be answered by machines only. Instead of using crowd-sourcing operators which give humans tasks to do, LLMs can perform tasks like extracting data from text, images, etc. much faster and often with better quality. However, any task done by an LLM still has a quality/time/cost trade-off and, similar to crowd-sourcing tasks, the quality increases the more one is willing to pay per task (e.g., by using a large vs small model, single vs multiple invocations, etc.). However, while this trade-off creates some similarities between PALIMPZEST and crowd-sourcing systems, PALIMPZEST's use of LLMs provides more degrees of freedom (e.g., the ability to generate code in seconds), creates new sets of challenges (e.g., token optimization), and does not have to deal with "lazy" workers, which was one of the key challenges of declarative crowd-based systems.

7 Conclusion

PALIMPZEST propels advancements in the field of AI workload optimization, providing a powerful yet user-friendly solution for managing complex AI-powered analytical queries. By employing a declarative language, PALIMPZEST simplifies the process of defining queries, allowing users to focus on the high-level logic of their applications without getting bogged down in the intricacies of underlying AI models and optimization strategies. PALIMPZEST seamlessly integrates logical and physical planning, ensuring that each query is executed in the most efficient manner possible. Furthermore, PALIMPZEST's runtime optimizations dynamically adapt to changing conditions, optimizing performance, quality, and cost-effectiveness in real time. Together, these features make PALIMPZEST an invaluable tool for developers and organizations aiming to take advantage of the full potential of modern AI capabilities efficiently and affordably. The simplicity of the PALIMPZEST's declarative language, combined with its intelligent and automatic planning and optimization capabilities, presents a new opportunity for the development of semantic analytics applications. This promises substantial improvements in both the speed and quality of AI-driven data processing tasks, while optimizing costs.

References

[1] Modal.com api. `https://modal.com`, 2024. Accessed: 2024-04-30.

[2] Ollama. `https://ollama.com`, 2024. Accessed: 2024-04-30.

[3] Together.ai. `https://together.ai`, 2024. Accessed: 2024-04-30.

[4] Michael R Anderson, Michael Cafarella, German Ros, and Thomas F Wenisch. Physical representation-based predicate optimization for a visual analytics database. In *2019 IEEE 35th International Conference on Data Engineering (ICDE)*, pages 1466–1477. IEEE, 2019.

[5] Simran Arora, Brandon Yang, Sabri Eyuboglu, Avanika Narayan, Andrew Hojel, Immanuel Trummer, and Christopher Ré. Language models enable simple systems for generating structured views of heterogeneous data lakes. *arXiv preprint arXiv:2304.09433*, 2023.

[6] Anonymous authors. CascadeServe: Unlocking model cascades for inference serving. Under submission, 2024.

[7] Philip A Bernstein. Applying model management to classical meta data problems. In *CIDR*, volume 2003, pages 209–220, 2003.

[8] Zhaowei Cai, Mohammad J. Saberian, and Nuno Vasconcelos. Learning complexity-aware cascades for deep pedestrian detection. *2015 IEEE International Conference on Computer Vision (ICCV)*, pages 3361–3369, 2015.

[9] Liwei Cao, Chen Huang, Daniel Cui Zhou, Yingwei Hu, T. Mamie Lih, Sara R. Savage, Karsten Krug, David J. Clark, Michael Schnaubelt, Lijun Chen, Felipe da Veiga Leprevost, Rodrigo Vargas Eguez, Weiming Yang, Jianbo Pan, Bo Wen, Yongchao Dou, Wen Jiang, Yuxing Liao, Zhiao Shi, Nadezhda V. Terekhanova, Song Cao, Rita Jui-Hsien Lu, Yize Li, Ruiyang Liu, Houxiang Zhu, Peter Ronning, Yige Wu, Matthew A. Wyczalkowski, Hariharan Easwaran, Ludmila Danilova, Arvind Singh Mer, Seungyeul Yoo, Joshua M. Wang, Wenke Liu, Benjamin Haibe-Kains, Mathangi Thiagarajan, Scott D. Jewell, Galen Hostetter, Chelsea J. Newton, Qing Kay Li, Michael H. Roehrl, David Fenyö, Pei Wang, Alexey I. Nesvizhskii, D.R. Mani, Gilbert S. Omenn, Emily S. Boja, Mehdi Mesri, Ana I. Robles, Henry Rodriguez, Oliver F. Bathe, Daniel W. Chan, Ralph H. Hruban, Li Ding, Bing Zhang, Hui Zhang, Mitual Amin, Eunkyung An, Christina Ayad, Thomas Bauer, Chet Birger, Michael J. Birrer, Simina M. Boca, William Bocik, Melissa Borucki, Shuang Cai, Steven A. Carr, Sandra Cerda, Huan Chen, Steven Chen, David Chesla, Arul M. Chinnaiyan, Antonio Colaprico, Sandra Cottingham, Magdalena Derejska, Saravana M. Dhanasekaran, Marcin J. Domagalski, Brian J. Druker, Elizabeth Duffy, Maureen A. Dyer, Nathan J. Edwards, Matthew J. Ellis, Jennifer Eschbacher, Alicia Francis, Jesse Francis, Stacey Gabriel, Nikolay Gabrovski, Johanna Gardner, Gad Getz, Michael A. Gillette, Charles A. Goldthwaite, Pamela Grady, Shuai Guo, Pushpa Hariharan, Tara Hiltke, Barbara Hindenach, Katherine A. Hoadley, Jasmine Huang, Corbin D. Jones, Karen A. Ketchum, Christopher R. Kinsinger, Jennifer M. Koziak, Katarzyna Kusnierz, Tao Liu, Jiang Long, David Mallery, Sailaja Mareedu, Ronald Matteotti, Nicollette Maunganidze, Peter B. McGarvey, Parham Minoo, Oxana V. Paklina, Amanda G. Paulovich, Samuel H. Payne, Olga Potapova, Barbara Pruetz, Liqun Qi, Nancy Roche, Karin D. Rodland, Daniel C. Rohrer, Eric E. Schadt, Alexey V. Shabunin, Troy Shelton, Yvonne Shutack, Shilpi Singh, Michael Smith, Richard D. Smith, Lori J. Sokoll, James Suh, Ratna R. Thangudu, Shirley X. Tsang, Ki Sung Um, Dana R. Valley, Negin Vatanian, Wenyi Wang, George D. Wilson, Maciej Wiznerowicz, Zhen Zhang, and Grace Zhao. Proteogenomic characterization of pancreatic ductal adenocarcinoma. *Cell*, 184(19):5031–5052.e26, September 2021.

[10] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

[11] Lingjiao Chen, Matei Zaharia, and James Zou. Frugalgpt: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.

[12] Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E Gonzalez, et al. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality. *See https://vicuna. lmsys. org (accessed 14 April 2023)*, 2(3):6, 2023.

[13] David J. Clark, Saravana M. Dhanasekaran, Francesca Petralia, Jianbo Pan, Xiaoyu Song, Yingwei Hu, Felipe da Veiga Leprevost, Boris Reva, Tung-Shing M. Lih, Hui-Yin Chang, Weiping Ma, Chen Huang, Christopher J. Ricketts, Lijun Chen, Azra Krek, Yize Li, Dmitry Rykunov, Qing Kay Li, Lin S. Chen, Umut Ozbek, Suhas Vasaikar, Yige Wu, Seungyeul Yoo, Shrabanti Chowdhury, Matthew A. Wyczalkowski, Jiayi Ji, Michael Schnaubelt, Andy Kong, Sunantha Sethuraman, Dmitry M. Avtonomov, Minghui Ao, Antonio Colaprico, Song Cao, Kyung-Cho Cho, Selim Kalayci, Shiyong Ma, Wenke Liu, Kelly Ruggles, Anna Calinawan, Zeynep H. Gümüş, Daniel Geiszler, Emily Kawaler, Guo Ci Teo, Bo Wen, Yuping Zhang, Sarah Keegan, Kai Li, Feng Chen, Nathan Edwards, Phillip M. Pierorazio, Xi Steven Chen, Christian P. Pavlovich, A. Ari Hakimi, Gabriel Brominski, James J. Hsieh, Andrzej Antczak, Tatiana Omelchenko, Jan Lubinski, Maciej Wiznerowicz, W. Marston Linehan, Christopher R. Kinsinger, Mathangi Thiagarajan, Emily S. Boja, Mehdi Mesri, Tara Hiltke, Ana I. Robles, Henry Rodriguez, Jiang Qian, David Fenyö, Bing Zhang, Li Ding, Eric Schadt, Arul M. Chinnaiyan, Zhen Zhang, Gilbert S. Omenn, Marcin Cieslik, Daniel W. Chan, Alexey I. Nesvizhskii, Pei Wang, Hui Zhang, Abdul Samad Hashimi, Alexander R. Pico, Alla Karpova, Alyssa Charamut, Amanda G. Paulovich, Amy M. Perou, Anna Malovannaya, Annette Marrero-Oliveras, Anupriya Agarwal, Barbara Hindenach, Barbara Pruetz, Beom-Jun Kim, Brian J. Druker, Chelsea J. Newton, Chet Birger, Corbin D. Jones, Cristina Tognon, D.R. Mani, Dana R. Valley, Daniel C. Rohrer, Daniel C. Zhou, Darlene Tansil, David Chesla, David Heiman, David Wheeler, Donghui Tan, Doug Chan, Emek Demir, Ewa Malc, Francesmary Modugno, Gaddy Getz, Galen Hostetter, George D. Wilson, Gerald W. Hart, Heng Zhu, Hongwei Liu, Houston Culpepper, Hua Sun, Hua Zhou, Jacob Day, James Suh, Jasmine

Huang, Jason McDermott, Jeffrey R. Whiteaker, Jeffrey W. Tyner, Jennifer Eschbacher, Jin Chen, John McGee, Jun Zhu, Karen A. Ketchum, Karin D. Rodland, Karl Clauser, Karna Robinson, Karsten Krug, Katherine A. Hoadley, Ki Sung Um, Kim Elburn, Kimberly Holloway, Liang-Bo Wang, Lili Blumenberg, Linda Hannick, Liqun Qi, Lori J. Sokoll, MacIntosh Cornwell, Marc Loriaux, Marcin J. Domagalski, Marina A. Gritsenko, Matthew Anderson, Matthew E. Monroe, Matthew J. Ellis, Maureen Dyer, Meenakshi Anurag, Meghan C. Burke, Melissa Borucki, Michael A. Gillette, Michael J. Birrer, Michael Lewis, Michael M. Ittmann, Michael Smith, Michael Vernon, Michelle Chaikin, Milan G. Chheda, Munziba Khan, Nancy Roche, Nathan J. Edwards, Negin Vatanian, Nicole Tignor, Noam Beckmann, Pamela Grady, Patricia Castro, Paul Piehowski, Peter B. McGarvey, Piotr Mieczkowski, Pushpa Hariharan, Qingsong Gao, Rajiv Dhir, Ramani Bhupendra Kothadia, Ratna R. Thangudu, Rebecca Montgomery, Reyka G. Jayasinghe, Richard D. Smith, Robert Edwards, Robert Zelt, Ross Bremner, Ruiyang Liu, Runyu Hong, Sailaja Mareedu, Samuel H. Payne, Sandra Cottingham, Sanford P. Markey, Scott D. Jewell, Shalin Patel, Shankha Satpathy, Shannon Richey, Sherri R. Davies, Shuang Cai, Simina M. Boca, Snehal Patil, Sohini Sengupta, Sonya Carter, Stacey Gabriel, Stefani N. Thomas, Stephanie De Young, Stephen E. Stein, Steven A. Carr, Steven M. Foltz, Sue Hilsenbeck, Tanya Krubit, Tao Liu, Tara Skelly, Thomas Westbrook, Uma Borate, Uma Velvulou, Vladislav A. Petyuk, William E. Bocik, Xi Chen, Yan Shi, Yifat Geffen, Yihao Lu, Ying Wang, Yosef Maruvka, Zhi Li, Zhiao Shi, and Zhidong Tu. Integrated proteogenomic characterization of clear cell renal cell carcinoma. *Cell*, 179(4):964–983.e31, October 2019.

[14] William W. Cohen. Enron email dataset, May 2015.

[15] LangChain Contributors. Langchain: Open source framework for building language models. `https://github.com/LangChain/langchain`, 2024. Accessed: 2024-04-18.

[16] Yongchao Dou, Emily A. Kawaler, Daniel Cui Zhou, Marina A. Gritsenko, Chen Huang, Lili Blumenberg, Alla Karpova, Vladislav A. Petyuk, Sara R. Savage, Shankha Satpathy, Wenke Liu, Yige Wu, Chia-Feng Tsai, Bo Wen, Zhi Li, Song Cao, Jamie Moon, Zhiao Shi, MacIntosh Cornwell, Matthew A. Wyczalkowski, Rosalie K. Chu, Suhas Vasaikar, Hua Zhou, Qingsong Gao, Ronald J. Moore, Kai Li, Sunantha Sethuraman, Matthew E. Monroe, Rui Zhao, David Heiman, Karsten Krug, Karl Clauser, Ramani Kothadia, Yosef Maruvka, Alexander R. Pico, Amanda E. Oliphant, Emily L. Hoskins, Samuel L. Pugh, Sean J.I. Beecroft, David W. Adams, Jonathan C. Jarman, Andy Kong, Hui-Yin Chang, Boris Reva, Yuxing Liao, Dmitry Rykunov, Antonio Colaprico, Xi Steven Chen, Andrzej Czekański, Marcin Jędryka, Rafał Matkowski, Maciej Wiznerowicz, Tara Hiltke, Emily Boja, Christopher R. Kinsinger, Mehdi Mesri, Ana I. Robles, Henry Rodriguez, David Mutch, Katherine Fuh, Matthew J. Ellis, Deborah DeLair, Mathangi Thiagarajan, D.R. Mani, Gad Getz, Michael Noble, Alexey I. Nesvizhskii, Pei Wang, Matthew L. Anderson, Douglas A. Levine, Richard D. Smith, Samuel H. Payne, Kelly V. Ruggles, Karin D. Rodland, Li Ding, Bing Zhang, Tao Liu, David Fenyö, Anupriya Agarwal, Meenakshi Anurag, Dmitry Avtonomov, Chet Birger, Michael J. Birrer, Simina M. Boca, William E. Bocik, Uma Borate, Melissa Borucki, Meghan C. Burke, Shuang Cai, Anna Calinawan, Steven A. Carr, Sonya Carter, Patricia Castro, Sandra Cerda, Michelle Chaikin, Daniel W. Chan, Doug Chan, Alyssa Charamut, Feng Chen, Jin Chen, Lijun Chen, Lin S. Chen, David Chesla, Milan G. Chheda, Arul M. Chinnaiyan, Shrabanti Chowdhury, Marcin P. Cieslik, David J. Clark, Sandra Cottingham, Houston Culpepper, Jacob Day, Stephanie De Young, Emek Demir, Saravana Mohan Dhanasekaran, Rajiv Dhir, Marcin J. Domagalski, Peter Dottino, Brian Druker, Elizabeth Duffy, Maureen Dyer, Nathan J. Edwards, Robert Edwards, Kim Elburn, Jayson B. Field, Alicia Francis, Stacey Gabriel, Yifat Geffen, Daniel Geiszler, Michael A. Gillette, Andrew K. Godwin, Pamela Grady, Linda Hannick, Pushpa Hariharan, Sue Hilsenbeck, Barbara Hindenach, Katherine A. Hoadley, Runyu Hong, Galen Hostetter, James J. Hsieh, Yingwei Hu, Michael M. Ittmann, Eric Jaehnig, Scott D. Jewell, Jiayi Ji, Corbin D. Jones, Renee Karabon, Karen A. Ketchum, Munziba Khan, Beom-Jun Kim, Azra Krek, Tanya Krubit, Chandan Kumar-Sinha, Felipe D. Leprevost, Michael Lewis, Qing Kay Li, Yize Li, Hongwei Liu, Jan Lubinski, Weiping Ma, Rashna Madan, Ewa Malc, Anna Malovannaya, Sailaja Mareedu, Sanford P. Markey, Annette Marrero-Oliveras, John Martignetti, Jason McDermott, Peter B. McGarvey, John McGee, Piotr Mieczkowski, Francesmary Modugno, Rebecca Montgomery, Chelsea J. Newton, Gilbert S. Omenn, Amanda G. Paulovich, Amy M. Perou, Francesca Petralia, Paul Piehowski, Larisa Polonskaya, Liqun Qi, Shannon Richey, Karna Robinson, Nancy Roche, Daniel C. Rohrer, Eric E. Schadt, Michael Schnaubelt, Yan Shi, Tara Skelly, Lori J. Sokoll, Xiaoyu Song, Stephen E. Stein, James Suh, Donghui Tan, Darlene Tansil, Guo Ci Teo, Ratna R. Thangudu, Cristina Tognon, Elie Traer, Jeffrey Tyner, Ki Sung Um, Dana R. Valley, Negin Vatanian, Pankaj Vats, Uma Velvulou, Michael Vernon, Liang-Bo Wang, Ying Wang, Alex Webster, Thomas Westbrook, David Wheeler, Jeffrey R. Whiteaker, George D. Wilson, Yuriy Zakhartsev, Robert Zelt, Hui Zhang, Yuping Zhang, Zhen Zhang, and Grace Zhao. Proteogenomic characterization of endometrial carcinoma. *Cell*, 180(4):729–748.e26, February 2020.

[17] Amber Feng, Michael J. Franklin, Donald Kossmann, Tim Kraska, Samuel Madden, Sukriti Ramesh, Andrew Wang, and Reynold Xin. Crowddb: Query processing with the VLDB crowd. *Proc. VLDB Endow.*, 4(12):1387–1390, 2011.

[18] Michael A. Gillette, Shankha Satpathy, Song Cao, Saravana M. Dhanasekaran, Suhas V. Vasaikar, Karsten Krug, Francesca Petralia, Yize Li, Wen-Wei Liang, Boris Reva, Azra Krek, Jiayi Ji, Xiaoyu Song, Wenke Liu, Runyu Hong, Lijun Yao, Lili Blumenberg, Sara R. Savage, Michael C. Wendl, Bo Wen, Kai Li, Lauren C. Tang, Melanie A. MacMullan, Shayan C. Avanessian, M. Harry Kane, Chelsea J. Newton, MacIntosh Cornwell, Ramani B. Kothadia, Weiping Ma, Seungyeul Yoo, Rahul Mannan, Pankaj Vats, Chandan Kumar-Sinha, Emily A. Kawaler, Tatiana Omelchenko, Antonio Colaprico, Yifat Geffen, Yosef E. Maruvka, Felipe da Veiga Leprevost, Maciej Wiznerowicz, Zeynep H. Gümüş, Rajwanth R. Veluswamy, Galen Hostetter, David I. Heiman, Matthew A. Wyczalkowski, Tara Hiltke, Mehdi Mesri, Christopher R. Kinsinger, Emily S. Boja, Gilbert S. Omenn, Arul M. Chinnaiyan, Henry Rodriguez, Qing Kay Li, Scott D. Jewell, Mathangi Thiagarajan, Gad Getz, Bing Zhang, David Fenyö, Kelly V. Ruggles, Marcin P. Cieslik, Ana I. Robles, Karl R. Clauser, Ramaswamy Govindan, Pei Wang, Alexey I. Nesvizhskii, Li Ding, D.R. Mani, Steven A. Carr, Alex Webster, Alicia Francis, Alyssa Charamut, Amanda G. Paulovich, Amy M. Perou, Andrew K. Godwin, Andrii Karnuta, Annette Marrero-Oliveras, Barbara Hindenach, Barbara Pruetz, Bartosz Kubisa, Brian J. Druker, Chet Birger, Corbin D. Jones, Dana R. Valley, Daniel C. Rohrer, Daniel Cui Zhou, Daniel W. Chan, David Chesla, David J. Clark, Dmitry Rykunov, Donghui Tan, Elena V. Ponomareva, Elizabeth Duffy, Eric J. Burks, Eric E. Schadt, Erik J. Bergstrom, Eugene S. Fedorov, Ewa Malc, George D. Wilson, Hai-Quan Chen, Halina M. Krzystek, Hongwei Liu, Houston Culpepper, Hua Sun, Hui Zhang, Jacob Day, James Suh, Jeffrey R. Whiteaker, Jennifer Eschbacher, John McGee, Karen A. Ketchum, Karin D. Rodland, Karna Robinson, Katherine A. Hoadley, Kei Suzuki, Ki Sung Um, Kim Elburn, Liang-Bo Wang, Lijun Chen, Linda Hannick, Liqun Qi, Lori J. Sokoll, Małgorzata Wojtyś, Marcin J. Domagalski, Marina A. Gritsenko, Mary B. Beasley, Matthew E. Monroe, Matthew J. Ellis, Maureen Dyer, Meghan C. Burke, Melissa Borucki, Meng-Hong Sun, Michael H. Roehrl, Michael J. Birrer, Michael Noble, Michael Schnaubelt, Michael Vernon, Michelle Chaikin, Mikhail Krotevich, Munziba Khan, Myvizhi Esai Selvan, Nancy Roche, Nathan J. Edwards, Negin Vatanian, Olga Potapova, Pamela Grady, Peter B. McGarvey, Piotr Mieczkowski, Pushpa Hariharan, Rashna Madan, Ratna R. Thangudu, Richard D. Smith, Robert J. Welsh, Robert Zelt, Rohit Mehra, Ronald Matteotti, Sailaja Mareedu, Samuel H. Payne, Sandra Cottingham, Sanford P. Markey, Seema Chugh, Shaleigh Smith, Shirley Tsang, Shuang Cai, Simina M. Boca, Sonya Carter, Stacey Gabriel, Stephanie De Young, Stephen E. Stein, Sunita Shankar, Tanya Krubit, Tao Liu, Tara Skelly, Thomas Bauer, Uma Velvulou, Umut Ozbek, Vladislav A. Petyuk, Volodymyr Sovenko, William E. Bocik, William W. Maggio, Xi Chen, Yan Shi, Yige Wu, Yingwei Hu, Yuxing Liao, Zhen Zhang, and Zhiao Shi. Proteogenomic characterization reveals therapeutic vulnerabilities in lung adenocarcinoma. *Cell*, 182(1):200–225.e35, July 2020.

[19] Jianping Gou, Baosheng Yu, Stephen J. Maybank, and Dacheng Tao. Knowledge distillation: A survey. *International Journal of Computer Vision*, 129(6):1789–1819, March 2021.

[20] Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. Minillm: Knowledge distillation of large language models. In *The Twelfth International Conference on Learning Representations*, 2023.

[21] Guidance AI Contributors. Guidance AI: Open source project for AI development. `https://github.com/guidance-ai/guidance`, 2023. Accessed: 2023-04-30.

[22] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

[23] Sirui Hong, Xiawu Zheng, Jonathan Chen, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, et al. Metagpt: Meta programming for multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023.

[24] Chen Huang, Lijun Chen, Sara R. Savage, Rodrigo Vargas Eguez, Yongchao Dou, Yize Li, Felipe da Veiga Leprevost, Eric J. Jaehnig, Jonathan T. Lei, Bo Wen, Michael Schnaubelt, Karsten Krug, Xiaoyu Song, Marcin Cieślik, Hui-Yin Chang, Matthew A. Wyczalkowski, Kai Li, Antonio Colaprico, Qing Kay Li, David J. Clark, Yingwei Hu, Liwei Cao, Jianbo Pan, Yuefan Wang, Kyung-Cho Cho, Zhiao Shi, Yuxing Liao, Wen Jiang, Meenakshi Anurag, Jiayi Ji, Seungyeul Yoo, Daniel Cui Zhou, Wen-Wei Liang, Michael Wendl, Pankaj Vats, Steven A. Carr, D.R. Mani, Zhen Zhang, Jiang Qian, Xi S. Chen, Alexander R. Pico, Pei Wang, Arul M. Chinnaiyan, Karen A. Ketchum, Christopher R. Kinsinger, Ana I. Robles, Eunkyung An, Tara Hiltke, Mehdi Mesri, Mathangi Thiagarajan, Alissa M. Weaver, Andrew G. Sikora, Jan Lubiński, Małgorzata Wierzbicka, Maciej Wiznerowicz, Shankha Satpathy, Michael A. Gillette, George Miles, Matthew J. Ellis, Gilbert S. Omenn, Henry Rodriguez, Emily S. Boja, Saravana M. Dhanasekaran, Li Ding, Alexey I. Nesvizhskii, Adel K. El-Naggar, Daniel W. Chan, Hui Zhang, Bing Zhang, Anupriya Agarwal, Matthew L. Anderson, Shayan C. Avanessian, Dmitry Avtonomov, Oliver F. Bathe, Chet Birger, Michael J. Birrer, Lili Blumenberg, William E. Bocik, Uma Borate, Melissa Borucki, Meghan C. Burke, Shuang Cai, Anna Pamela Calinawan, Sandra Cerda, Alyssa Charamut, Lin S. Chen, Shrabanti Chowdhury, Karl R. Clauser, Houston Culpepper, Tomasz Czernicki, Fulvio D'Angelo, Jacob Day, Stephanie De Young, Emek Demir, Fei Ding, Marcin J. Domagalski, Joseph C. Dort, Brian Druker, Elizabeth Duffy, Maureen Dyer, Nathan J. Edwards, Kimberly Elburn, Tatiana S. Ermakova, David Fenyo, Renata Ferrarotto, Alicia

Francis, Stacey Gabriel, Luciano Garofano, Yifat Geffen, Gad Getz, Charles A. Goldthwaite, Linda I. Hannick, Pushpa Hariharan, David N. Hayes, David Heiman, Barbara Hindenach, Katherine A. Hoadley, Galen Hostetter, Martin Hyrcza, Scott D. Jewell, Corbin D. Jones, M. Harry Kane, Alicia Karz, Ramani B. Kothadia, Azra Krek, Chandan Kumar-Sinha, Tao Liu, Hongwei Liu, Weiping Ma, Ewa Malc, Anna Malovannaya, Sailaja Mareedu, Sanford P. Markey, Annette Marrero-Oliveras, Nicollette Maunganidze, Jason E. McDermott, Peter B. McGarvey, John McGee, Piotr Mieczkowski, Simona Migliozzi, Rebecca Montgomery, Chelsea J. Newton, Umut Ozbek, Amanda G. Paulovich, Samuel H. Payne, Dimitar Dimitrov Pazardzhikliev, Amy M. Perou, Francesca Petralia, Lyudmila Petrenko, Paul D. Piehowski, Dmitris Placantonakis, Larisa Polonskaya, Elena V. Ponomareva, Olga Potapova, Liqun Qi, Ning Qu, Shakti Ramkissoon, Boris Reva, Shannon Richey, Karna Robinson, Nancy Roche, Karin Rodland, Daniel C. Rohrer, Dmitry Rykunov, Eric E. Schadt, Yan Shi, Yvonne Shutack, Shilpi Singh, Tara Skelly, Richard Smith, Lori J. Sokoll, Jakub Stawicki, Stephen E. Stein, James Suh, Wojciech Szopa, Dave Tabor, Donghui Tan, Darlene Tansil, Guo Ci Teo, Ratna R. Thangudu, Cristina Tognon, Elie Traer, Shirley Tsang, Jeffrey Tyner, Ki Sung Um, Dana R. Valley, Lyubomir Valkov Vasilev, Negin Vatanian, Uma Velvulou, Michael Vernon, Thomas F. Westbrook, Jeffrey R. Whiteaker, Yige Wu, Midie Xu, Lijun Yao, Xinpei Yi, Fengchao Yu, Kakhaber Zaalishvili, Yuriy Zakhartsev, Robert Zelt, Grace Zhao, and Jun Zhu. Proteogenomic insights into the biology and treatment of hpv-negative head and neck squamous cell carcinoma. *Cancer Cell*, 39(3):361–379.e16, March 2021.

[25] Daniel Kang, John Emmons, Firas Abuzaid, Peter Bailis, and Matei Zaharia. Noscope: optimizing neural network queries over video at scale. *Proc. VLDB Endow.*, 10(11):1586–1597, aug 2017.

[26] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Moazam, et al. Dspy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.

[27] Karsten Krug, Eric J. Jaehnig, Shankha Satpathy, Lili Blumenberg, Alla Karpova, Meenakshi Anurag, George Miles, Philipp Mertins, Yifat Geffen, Lauren C. Tang, David I. Heiman, Song Cao, Yosef E. Maruvka, Jonathan T. Lei, Chen Huang, Ramani B. Kothadia, Antonio Colaprico, Chet Birger, Jarey Wang, Yongchao Dou, Bo Wen, Zhiao Shi, Yuxing Liao, Maciej Wiznerowicz, Matthew A. Wyczalkowski, Xi Steven Chen, Jacob J. Kennedy, Amanda G. Paulovich, Mathangi Thiagarajan, Christopher R. Kinsinger, Tara Hiltke, Emily S. Boja, Mehdi Mesri, Ana I. Robles, Henry Rodriguez, Thomas F. Westbrook, Li Ding, Gad Getz, Karl R. Clauser, David Fenyö, Kelly V. Ruggles, Bing Zhang, D.R. Mani, Steven A. Carr, Matthew J. Ellis, Michael A. Gillette, Shayan C. Avanessian, Shuang Cai, Daniel Chan, Xian Chen, Nathan J. Edwards, Andrew N. Hoofnagle, M. Harry Kane, Karen A. Ketchum, Eric Kuhn, Douglas A. Levine, Shunqiang Li, Daniel C. Liebler, Tao Liu, Jingqin Luo, Subha Madhavan, Chris Maher, Jason E. McDermott, Peter B. McGarvey, Mauricio Oberti, Akhilesh Pandey, Samuel H. Payne, David F. Ransohoff, Robert C. Rivers, Karin D. Rodland, Paul Rudnick, Melinda E. Sanders, Kenna M. Shaw, Ie-Ming Shih, Robbert J.C. Slebos, Richard D. Smith, Michael Snyder, Stephen E. Stein, David L. Tabb, Ratna R. Thangudu, Stefani Thomas, Yue Wang, Forest M. White, Jeffrey R. Whiteaker, Gordon A. Whiteley, Hui Zhang, Zhen Zhang, Yingming Zhao, Heng Zhu, and Lisa J. Zimmerman. Proteogenomic landscape of breast cancer tumorigenesis and targeted therapy. *Cell*, 183(5):1436–1456.e31, November 2020.

[28] Michael Kuchnik, Virginia Smith, and George Amvrosiadis. Validating large language models with relm. *Proceedings of Machine Learning and Systems*, 5, 2023.

[29] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pages 19274–19286. PMLR, 2023.

[30] Yize Li, Yongchao Dou, Felipe Da Veiga Leprevost, Yifat Geffen, Anna P. Calinawan, François Aguet, Yo Akiyama, Shankara Anand, Chet Birger, Song Cao, Rekha Chaudhary, Padmini Chilappagari, Marcin Cieslik, Antonio Colaprico, Daniel Cui Zhou, Corbin Day, Marcin J. Domagalski, Myvizhi Esai Selvan, David Fenyö, Steven M. Foltz, Alicia Francis, Tania Gonzalez-Robles, Zeynep H. Gümüş, David Heiman, Michael Holck, Runyu Hong, Yingwei Hu, Eric J. Jaehnig, Jiayi Ji, Wen Jiang, Lizabeth Katsnelson, Karen A. Ketchum, Robert J. Klein, Jonathan T. Lei, Wen-Wei Liang, Yuxing Liao, Caleb M. Lindgren, Weiping Ma, Lei Ma, Michael J. MacCoss, Fernanda Martins Rodrigues, Wilson McKerrow, Ngoc Nguyen, Robert Oldroyd, Alexander Pilozzi, Pietro Pugliese, Boris Reva, Paul Rudnick, Kelly V. Ruggles, Dmitry Rykunov, Sara R. Savage, Michael Schnaubelt, Tobias Schraink, Zhiao Shi, Deepak Singhal, Xiaoyu Song, Erik Storrs, Nadezhda V. Terekhanova, Ratna R. Thangudu, Mathangi Thiagarajan, Liang-Bo Wang, Joshua M. Wang, Ying Wang, Bo Wen, Yige Wu, Matthew A. Wyczalkowski, Yi Xin, Lijun Yao, Xinpei Yi, Hui Zhang, Qing Zhang, Maya Zuhl, Gad Getz, Li Ding, Alexey I. Nesvizhskii, Pei Wang, Ana I. Robles, Bing Zhang, Samuel H. Payne, Alexander J. Lazar, Amanda G. Paulovich, Antonio Colaprico, Antonio Iavarone, Arul M. Chinnaiyan, Brian J. Druker, Chandan Kumar-Sinha, Chelsea J. Newton, Chen Huang, D.R. Mani, Richard D. Smith, Emily Huntsman, Eric E. Schadt, Eunkyung An, Francesca Petralia, Galen Hostetter, Gilbert S. Omenn, Hanbyul Cho, Henry Rodriguez, Hui Zhang, Iga Kolodziejczak, Jared L. Johnson, Jasmin Bavarva, Jimin Tan, Karin D. Rodland, Karl R. Clauser, Karsten Krug, Lewis C. Cantley, Maciej Wiznerowicz, Matthew J. Ellis, Meenakshi Anurag, Mehdi Mesri, Michael A. Gillette, Michael J. Birrer,

Michele Ceccarelli, Saravana M. Dhanasekaran, Nathan Edwards, Nicole Tignor, Özgün Babur, Pietro Pugliese, Sara J.C. Gosline, Scott D. Jewell, Shankha Satpathy, Shrabanti Chowdhury, Stephan Schürer, Steven A. Carr, Tao Liu, Tara Hiltke, Tomer M. Yaron, Vasileios Stathias, Wenke Liu, Xu Zhang, Yizhe Song, Zhen Zhang, and Daniel W. Chan. Proteogenomic data and resources for pan-cancer analysis. *Cancer Cell*, 41(8):1397–1406, August 2023.

[31] Yize Li, Yongchao Dou, Felipe Da Veiga Leprevost, Yifat Geffen, Anna P Calinawan, François Aguet, Yo Akiyama, Shankara Anand, Chet Birger, Song Cao, et al. Proteogenomic data and resources for pan-cancer analysis. *Cancer cell*, 41(8):1397–1406, 2023.

[32] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097, December 2022.

[33] Yiming Lin, Madelon Hulsebos, Ruiying Ma, Shreya Shankar, Sepanta Zeigham, Aditya G Parameswaran, and Eugene Wu. Towards accurate and efficient document analytics with large language models. *arXiv preprint arXiv:2405.04674*, 2024.

[34] Jerry Liu. LlamaIndex, 11 2022.

[35] Shu Liu, Asim Biswal, Audrey Cheng, Xiangxi Mo, Shiyi Cao, Joseph E Gonzalez, Ion Stoica, and Matei Zaharia. Optimizing llm queries in relational workloads. *arXiv preprint arXiv:2403.05821*, 2024.

[36] Xiaoxuan Liu, Lanxiang Hu, Peter Bailis, Ion Stoica, Zhijie Deng, Alvin Cheung, and Hao Zhang. Online speculative decoding. *arXiv preprint arXiv:2310.07177*, 2023.

[37] Adam Marcus, David R. Karger, Samuel Madden, Rob Miller, and Sewoong Oh. Counting with the crowd. *Proc. VLDB Endow.*, 6(2):109–120, 2012.

[38] Adam Marcus, Eugene Wu, Samuel Madden, and Robert C. Miller. Crowdsourced databases: Query processing with people. In *Fifth Biennial Conference on Innovative Data Systems Research, CIDR 2011, Asilomar, CA, USA, January 9-12, 2011, Online Proceedings*, pages 211–214. www.cidrdb.org, 2011.

[39] Jason E. McDermott, Osama A. Arshad, Vladislav A. Petyuk, Yi Fu, Marina A. Gritsenko, Therese R. Clauss, Ronald J. Moore, Athena A. Schepmoes, Rui Zhao, Matthew E. Monroe, Michael Schnaubelt, Chia-Feng Tsai, Samuel H. Payne, Chen Huang, Liang-Bo Wang, Steven Foltz, Matthew Wyczalkowski, Yige Wu, Ehwang Song, Molly A. Brewer, Mathangi Thiagarajan, Christopher R. Kinsinger, Ana I. Robles, Emily S. Boja, Henry Rodriguez, Daniel W. Chan, Bing Zhang, Zhen Zhang, Li Ding, Richard D. Smith, Tao Liu, and Karin D. Rodland. Proteogenomic characterization of ovarian hgsc implicates mitotic kinases, replication stress in observed chromosomal instability. *Cell Reports Medicine*, 1(1):100004, April 2020.

[40] OpenAI. Openai api. `https://platform.openai.com`, 2024. Accessed: 2024-04-30.

[41] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.

[42] Hyunjung Park, Richard Pang, Aditya G. Parameswaran, Hector Garcia-Molina, Neoklis Polyzotis, and Jennifer Widom. Deco: A system for declarative crowdsourcing. *Proc. VLDB Endow.*, 5(12):1990–1993, 2012.

[43] Pratyush Patel, Esha Choukse, Chaojie Zhang, Íñigo Goiri, Aashaka Shah, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative llm inference using phase splitting. *arXiv preprint arXiv:2311.18677*, 2023.

[44] Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. Gorilla: Large language model connected with massive apis. *arXiv preprint arXiv:2305.15334*, 2023.

[45] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36, 2024.

[46] Vijayshankar Raman and Joseph M. Hellerstein. Potter's wheel: An interactive data cleaning system. In *Proceedings of the 27th International Conference on Very Large Data Bases*, VLDB '01, page 381–390, San Francisco, CA, USA, 2001. Morgan Kaufmann Publishers Inc.

[47] Alexander Ratner, Stephen H. Bach, Henry Ehrenberg, Jason Fries, Sen Wu, and Christopher Ré. Snorkel: rapid training data creation with weak supervision. *Proceedings of the VLDB Endowment*, 11(3):269–282, November 2017.

[48] Shankha Satpathy, Karsten Krug, Pierre M. Jean Beltran, Sara R. Savage, Francesca Petralia, Chandan Kumar-Sinha, Yongchao Dou, Boris Reva, M. Harry Kane, Shayan C. Avanessian, Suhas V. Vasaikar, Azra Krek, Jonathan T. Lei, Eric J. Jaehnig, Tatiana Omelchenko, Yifat Geffen, Erik J. Bergstrom, Vasileios Stathias, Karen E. Christianson, David I. Heiman, Marcin P. Cieslik, Song Cao, Xiaoyu Song, Jiayi Ji, Wenke Liu, Kai Li, Bo Wen, Yize Li, Zeynep H. Gümüş, Myvizhi Esai Selvan, Rama Soundararajan, Tanvi H. Visal, Maria G. Raso, Edwin Roger Parra, Özgün Babur, Pankaj Vats, Shankara Anand, Tobias Schraink, MacIntosh Cornwell, Fernanda Martins Rodrigues, Houxiang Zhu, Chia-Kuei Mo, Yuping Zhang, Felipe da Veiga Leprevost, Chen Huang, Arul M. Chinnaiyan, Matthew A. Wyczalkowski, Gilbert S. Omenn, Chelsea J. Newton, Stephan Schurer, Kelly V. Ruggles, David Fenyö, Scott D. Jewell, Mathangi Thiagarajan, Mehdi Mesri, Henry Rodriguez, Sendurai A. Mani, Namrata D. Udeshi, Gad Getz, James Suh, Qing Kay Li, Galen Hostetter, Paul K. Paik, Saravana M. Dhanasekaran, Ramaswamy Govindan, Li Ding, Ana I. Robles, Karl R. Clauser, Alexey I. Nesvizhskii, Pei Wang, Steven A. Carr, Bing Zhang, D.R. Mani, Michael A. Gillette, Alex Green, Alfredo Molinolo, Alicia Francis, Amanda G. Paulovich, Andrii Karnuta, Antonio Colaprico, Barbara Hindenach, Barbara L. Pruetz, Bartosz Kubisa, Brian J. Druker, Carissa Huynh, Charles A. Goldthwaite, Chet Birger, Christopher R. Kinsinger, Corbin D. Jones, Dan Rohrer, Dana R. Valley, Daniel W. Chan, David Chesla, Donna Hansel, Elena V. Ponomareva, Elizabeth Duffy, Eric Burks, Eric E. Schadt, Eugene S. Fedorov, Eunkyung An, Fei Ding, George D. Wilson, Harsh Batra, Hui Zhang, Jennifer E. Maas, Jennifer Eschbacher, Karen A. Ketchum, Karin D. Rodland, Katherine A. Hoadley, Kei Suzuki, Ki Sung Um, Liqun Qi, Lori Bernard, Maciej Wiznerowicz, Małgorzata Wojtyś, Marcin J. Domagalski, Matthew J. Ellis, Maureen A. Dyer, Melissa Borucki, Meenakshi Anurag, Michael J. Birrer, Midie Xu, Mikhail Krotevich, Nancy Roche, Nathan J. Edwards, Negin Vatanian, Neil R. Mucci, Nicollette Maunganidze, Nikolay Gabrovski, Olga Potapova, Oluwole Fadare, Pamela Grady, Peter B. McGarvey, Pushpa Hariharan, Ratna R. Thangudu, Rebecca Montgomery, Renganayaki Pandurengan, Richard D. Smith, Robert J. Welsh, Sailaja Mareedu, Samuel H. Payne, Sandra Cottingham, Shilpi Singh, Shirley X. Tsang, Shuang Cai, Stacey Gabriel, Tao Liu, Tara Hiltke, Tanmayi Vashist, Thomas Bauer, Volodymyr Sovenko, Warren G. Tourtellotte, Weiping Ma, William Bocik, Wohaib Hasan, Xiaojun Jing, Ximing Tang, Yuxing Liao, Yvonne, Shutack, Zhen Zhang, and Ziad Hanhan. A proteogenomic portrait of lung squamous cell carcinoma. *Cell*, 184(16):4348–4371.e40, August 2021.

[49] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36, 2024.

[50] Benjamin Frederick Spector and Christopher Re. Accelerating llm inference with staged speculative decoding. In *Workshop on Efficient Systems for Foundation Models@ ICML2023*, 2023.

[51] Yi Sun, Xiaogang Wang, and Xiaoou Tang. Deep convolutional network cascade for facial point detection. *2013 IEEE Conference on Computer Vision and Pattern Recognition*, pages 3476–3483, 2013.

[52] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford alpaca: An instruction-following llama model. `https://github.com/tatsu-lab/stanford_alpaca`, 2023.

[53] Matthias Urban and Carsten Binnig. Caesura: Language models as multi-modal query planners. *arXiv preprint arXiv:2308.03424*, 2023.

[54] Suhas Vasaikar, Chen Huang, Xiaojing Wang, Vladislav A. Petyuk, Sara R. Savage, Bo Wen, Yongchao Dou, Yun Zhang, Zhiao Shi, Osama A. Arshad, Marina A. Gritsenko, Lisa J. Zimmerman, Jason E. McDermott, Therese R. Clauss, Ronald J. Moore, Rui Zhao, Matthew E. Monroe, Yi-Ting Wang, Matthew C. Chambers, Robbert J.C. Slebos, Ken S. Lau, Qianxing Mo, Li Ding, Matthew Ellis, Mathangi Thiagarajan, Christopher R. Kinsinger, Henry Rodriguez, Richard D. Smith, Karin D. Rodland, Daniel C. Liebler, Tao Liu, Bing Zhang, Akhilesh Pandey, Amanda Paulovich, Andrew Hoofnagle, D.R. Mani, Daniel W. Chan, David F. Ransohoff, David Fenyo, David L. Tabb, Douglas A. Levine, Emily S. Boja, Eric Kuhn, Forest M. White, Gordon A. Whiteley, Heng Zhu, Hui Zhang, Ie-Ming Shih, Jasmin Bavarva, Jeffrey Whiteaker, Karen A. Ketchum, Karl R. Clauser, Kelly Ruggles, Kimberly Elburn, Linda Hannick, Mark Watson, Mauricio Oberti, Mehdi Mesri, Melinda E. Sanders, Melissa Borucki, Michael A. Gillette, Michael Snyder, Nathan J. Edwards, Negin Vatanian, Paul A. Rudnick, Peter B. McGarvey, Philip Mertins, R. Reid Townsend, Ratna R. Thangudu, Robert C. Rivers, Samuel H. Payne, Sherri R. Davies, Shuang Cai, Stephen E. Stein, Steven A. Carr, Steven J. Skates, Subha Madhavan, Tara Hiltke, Xian Chen, Yingming Zhao, Yue Wang, and Zhen Zhang. Proteogenomic analysis of human colon cancer reveals new therapeutic opportunities. *Cell*, 177(4):1035–1049.e19, May 2019.

[55] Liang-Bo Wang, Alla Karpova, Marina A. Gritsenko, Jennifer E. Kyle, Song Cao, Yize Li, Dmitry Rykunov, Antonio Colaprico, Joseph H. Rothstein, Runyu Hong, Vasileios Stathias, MacIntosh Cornwell, Francesca Petralia, Yige Wu, Boris Reva, Karsten Krug, Pietro Pugliese, Emily Kawaler, Lindsey K. Olsen, Wen-Wei Liang, Xiaoyu Song, Yongchao Dou, Michael C. Wendl, Wagma Caravan, Wenke Liu, Daniel Cui Zhou, Jiayi Ji, Chia-Feng Tsai, Vladislav A. Petyuk, Jamie Moon, Weiping Ma, Rosalie K. Chu, Karl K. Weitz, Ronald J. Moore, Matthew E.

Monroe, Rui Zhao, Xiaolu Yang, Seungyeul Yoo, Azra Krek, Alexis Demopoulos, Houxiang Zhu, Matthew A. Wyczalkowski, Joshua F. McMichael, Brittany L. Henderson, Caleb M. Lindgren, Hannah Boekweg, Shuangjia Lu, Jessika Baral, Lijun Yao, Kelly G. Stratton, Lisa M. Bramer, Erika Zink, Sneha P. Couvillion, Kent J. Bloodsworth, Shankha Satpathy, Weiva Sieh, Simina M. Boca, Stephan Schürer, Feng Chen, Maciej Wiznerowicz, Karen A. Ketchum, Emily S. Boja, Christopher R. Kinsinger, Ana I. Robles, Tara Hiltke, Mathangi Thiagarajan, Alexey I. Nesvizhskii, Bing Zhang, D.R. Mani, Michele Ceccarelli, Xi S. Chen, Sandra L. Cottingham, Qing Kay Li, Albert H. Kim, David Fenyö, Kelly V. Ruggles, Henry Rodriguez, Mehdi Mesri, Samuel H. Payne, Adam C. Resnick, Pei Wang, Richard D. Smith, Antonio Iavarone, Milan G. Chheda, Jill S. Barnholtz-Sloan, Karin D. Rodland, Tao Liu, Li Ding, Anupriya Agarwal, Mitual Amin, Eunkyung An, Matthew L. Anderson, David W. Andrews, Thomas Bauer, Chet Birger, Michael J. Birrer, Lili Blumenberg, William E. Bocik, Uma Borate, Melissa Borucki, Meghan C. Burke, Shuang Cai, Anna P. Calinawan, Steven A. Carr, Sandra Cerda, Daniel W. Chan, Alyssa Charamut, Lin S. Chen, David Chesla, Arul M. Chinnaiyan, Shrabanti Chowdhury, Marcin P. Cieślik, David J. Clark, Houston Culpepper, Tomasz Czernicki, Fulvio D'Angelo, Jacob Day, Stephanie De Young, Emek Demir, Saravana Mohan Dhanasekaran, Rajiv Dhir, Marcin J. Domagalski, Brian Druker, Elizabeth Duffy, Maureen Dyer, Nathan J. Edwards, Robert Edwards, Kimberly Elburn, Matthew J. Ellis, Jennifer Eschbacher, Alicia Francis, Stacey Gabriel, Nikolay Gabrovski, Luciano Garofano, Gad Getz, Michael A. Gillette, Andrew K. Godwin, Denis Golbin, Ziad Hanhan, Linda I. Hannick, Pushpa Hariharan, Barbara Hindenach, Katherine A. Hoadley, Galen Hostetter, Chen Huang, Eric Jaehnig, Scott D. Jewell, Nan Ji, Corbin D. Jones, Alcida Karz, Wojciech Kaspera, Lyndon Kim, Ramani B. Kothadia, Chandan Kumar-Sinha, Jonathan Lei, Felipe D. Leprevost, Kai Li, Yuxing Liao, Jena Lilly, Hongwei Liu, Jan Lubínski, Rashna Madan, William Maggio, Ewa Malc, Anna Malovannaya, Sailaja Mareedu, Sanford P. Markey, Annette Marrero-Oliveras, Nina Martinez, Nicollette Maunganidze, Jason E. McDermott, Peter B. McGarvey, John McGee, Piotr Mieczkowski, Simona Migliozzi, Francesmary Modugno, Rebecca Montgomery, Chelsea J. Newton, Gilbert S. Omenn, Umut Ozbek, Oxana V. Paklina, Amanda G. Paulovich, Amy M. Perou, Alexander R. Pico, Paul D. Piehowski, Dimitris G. Placantonakis, Larisa Polonskaya, Olga Potapova, Barbara Pruetz, Liqun Qi, Shakti Ramkissoon, Adam Resnick, Shannon Richey, Gregory Riggins, Karna Robinson, Nancy Roche, Daniel C. Rohrer, Brian R. Rood, Larissa Rossell, Sara R. Savage, Eric E. Schadt, Yan Shi, Zhiao Shi, Yvonne Shutack, Shilpi Singh, Tara Skelly, Lori J. Sokoll, Jakub Stawicki, Stephen E. Stein, James Suh, Wojciech Szopa, Dave Tabor, Donghui Tan, Darlene Tansil, Ratna R. Thangudu, Cristina Tognon, Elie Traer, Shirley Tsang, Jeffrey Tyner, Ki Sung Um, Dana R. Valley, Suhas Vasaikar, Negin Vatanian, Uma Velvulou, Michael Vernon, Weiqing Wan, Junmei Wang, Alex Webster, Bo Wen, Jeffrey R. Whiteaker, George D. Wilson, Yuriy Zakhartsev, Robert Zelt, Hui Zhang, Liwei Zhang, Zhen Zhang, Grace Zhao, and Jun Zhu. Proteogenomic and metabolomic characterization of human glioblastoma. *Cancer Cell*, 39(4):509–528.e20, April 2021.

[56] Brandon T Willard and Rémi Louf. Efficient guided generation for llms. *arXiv preprint arXiv:2307.09702*, 2023.

[57] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation framework. *arXiv preprint arXiv:2308.08155*, 2023.

[58] Zongheng Yang, Zhanghao Wu, Michael Luo, Wei-Lin Chiang, Romil Bhardwaj, Woosuk Kwon, Siyuan Zhuang, Frank Sifei Luan, Gautam Mittal, Scott Shenker, and Ion Stoica. Skypilot: An intercloud broker for sky computing. In *Symposium on Networked Systems Design and Implementation*, 2023.

[59] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[60] Zihao Ye, Ruihang Lai, Bo-Ru Lu, Chien-Yu Lin, Size Zheng, Lequn Chen, Tianqi Chen, and Luis Ceze. Cascade inference: Memory bandwidth efficient shared prefix batch decoding. `https://flashinfer.ai/2024/02/02/cascade-inference.html`, February 2024.

[61] Matei Zaharia, Omar Khattab, Lingjiao Chen, Jared Quincy Davis, Heather Miller, Chris Potts, James Zou, Michael Carbin, Jonathan Frankle, Naveen Rao, and Ali Ghodsi. The shift from models to compound ai systems. `https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems/`, 2024.

[62] Jiahao Zhang, Haiyang Zhang, Dongmei Zhang, Yong Liu, and Shen Huang. End-to-end beam retrieval for multi-hop question answering. In *2024 Annual Conference of the North American Chapter of the Association for Computational Linguistics*, 2024.

[63] Siyan Zhao, Daniel Israel, Guy Van den Broeck, and Aditya Grover. Prepacking: A simple method for fast prefilling and increased throughput in large language models. *arXiv preprint arXiv:2404.09529*, 2024.

[64] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. Efficiently programming large language models using sglang. *arXiv preprint arXiv:2312.07104*, 2023.

Appendix

Evaluation Plan Details

In Subsection 5.3 we highlight three plans (PLAN 1, 2, and 3), which provide useful performance trade-offs relative to baseline plans. The implementation details for each of these plans are described below.

PLAN 1: uses Mixtral-8x7B to convert each `TextFile` to the `Email` schema. It then uses Mixtral-8x7B again to apply the filter which checks whether or not the email refers to a fraudulent investment scheme. Finally, this plan uses gpt-3.5-turbo to determine whether or not the email is referring to a news article or is written by someone outside of Enron. A depiction of the plan is shown below:

```
0. MarshalAndScanDataOp -> File

1. File -> InduceFromCandidateOp -> TextFile
   Using hardcoded function
   (contents,filena...) -> (contents,filena...)

2. TextFile -> InduceFromCandidateOp -> Email
   Using Model.MIXTRAL
   Token budget: 1.0
   Query strategy: QueryStrategy.BONDED_WITH_FALLBACK
   (contents,filena...) -> (contents,filena...)

3. Email -> FilterCandidateOp -> Email
   Using Model.MIXTRAL
   Filter: "The email refers to a fraudulent scheme (i.e., "Raptor", ...)"
   (contents,filena...) -> (contents,filena...)

4. Email -> FilterCandidateOp -> Email
   Using Model.GPT_3_5
   Filter: "The email is not quoting from a news article..."
   (contents,filena...) -> (contents,filena...)
```

PLAN 2: uses gpt-3.5-turbo (with a token budget allowing it to process up to 50% of the listing text) to convert each `RealEstateListingFiles` to the `TextRealEstateListing` schema. It then applies the UDF filters which check whether or not the listing is within two miles of MIT and in the user's price range. The plan then uses the gpt-4-vision-preview model to convert the schema to an `ImageRealEstateListing` (extracting the "modern and attractive" and "natural sunglight" attributes), before finally applying gpt-3.5-turbo to filter based on these attributes. A depiction of the plan is shown below:

```
0. MarshalAndScanDataOp -> RealEstateListingFiles

1. RealEstateListingFiles -> InduceFromCandidateOp -> TextRealEstateListing
   Using Model.GPT_3_5
   Token budget: 0.5
   Query strategy: QueryStrategy.BONDED_WITH_FALLBACK
   (image_contents,...) -> (address,image_c...)

2. TextRealEstateListing -> FilterCandidateOp -> TextRealEstateListing
   Using None
   Filter: "<function get_logical_tree.<locals>.within_two_miles_of_mit"
   (address,image_c...) -> (address,image_c...)

3. TextRealEstateListing -> FilterCandidateOp -> TextRealEstateListing
   Using None
   Filter: "<function get_logical_tree.<locals>.in_price_range"
   (address,image_c...) -> (address,image_c...)

4. TextRealEstateListing -> InduceFromCandidateOp -> ImageRealEstateListing
   Using Model.GPT_4V
   Token budget: 1.0
   Query strategy: QueryStrategy.BONDED_WITH_FALLBACK
   (address,image_c...) -> (has_natural_sun...)
```

```
5. ImageRealEstateListing -> FilterCandidateOp -> ImageRealEstateListing
   Using Model.GPT_3_5
   Filter: "The interior is modern and attractive, and has lots of natural sunlight"
   (has_natural_sun...) -> (has_natural_sun...)
```

PLAN 3: uses Mixtral-8x7B to filter for the subset of tables which contain patient age information. It then uses Mixtral-8x7B once again (with a token budget allowing it to process up to 90% of the input table data) to convert each input `Table` to the desired output `CaseData` schema. A depiction of the plan is shown below:

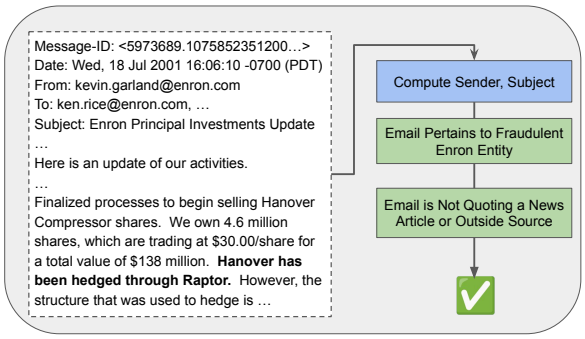
```
0. MarshalAndScanDataOp -> File

1. File -> InduceFromCandidateOp -> XLSFile
   Using hardcoded function
   (contents,filena...) -> (contents,filena...)

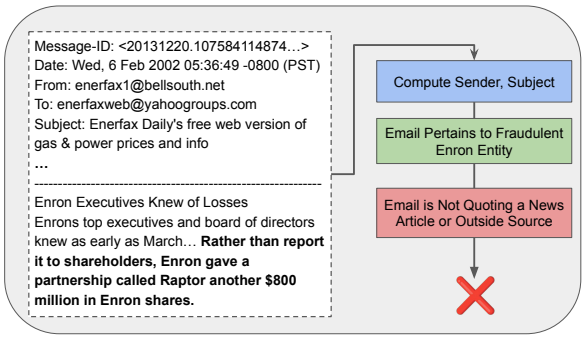
2. XLSFile -> InduceFromCandidateOp -> Table
   Using hardcoded function
   (contents,filena...) -> (filename,header...)

3. Table -> FilterCandidateOp -> Table
   Using Model.MIXTRAL
   Filter: "The rows of the table contain the patient age"
   (filename,header...) -> (filename,header...)

4. Table -> InduceFromCandidateOp -> CaseData
   Using Model.MIXTRAL
   Token budget: 0.9
   Query strategy: QueryStrategy.BONDED_WITH_FALLBACK
   (filename,header...) -> (age_at_diagnosi...)
```



(a) Example of a positive entry in the Legal Discovery workload. The email meets both criteria because it (1) pertains to a fraudulent entity and (2) is not quoting a news article or outside source.



(b) Example of a negative entry in the Legal Discovery workload. This email does not meet the criteria set by the user's search because the email is quoting from a news article.

Figure 8: A positive and negative examples in the Legal Discovery workload.

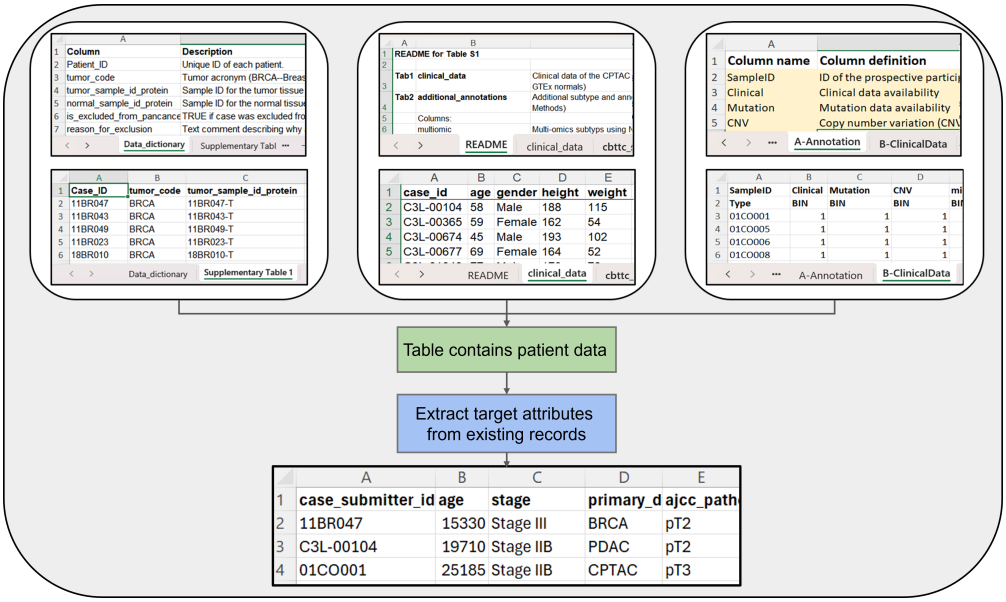


Figure 9: Example of the Medical Schema Matching workload. Three spreadsheets contain several tables and PALIMPZEST (1) filters for tables that relate to patient data and (2) matches and extracts the relevant attributes, consolidating a single table with a target schema.

```python
import palimpzest as pz

class CaseData(pz.Schema):
    """An individual row extracted from a table containing medical study data."""
    case_submitter_id = pz.Field(desc="The ID of the case")
    age_at_diagnosis = pz.Field(desc="The age of the patient...", required=False)
    race = pz.Field(desc="An arbitrary classification of a...", required=False)
    ethnicity = pz.Field(desc="Whether an individual...", required=False)
    gender = pz.Field(desc="Text designations that identify gender.", required=False)
    vital_status = pz.Field(desc="The vital status of the patient", required=False)
    ajcc_pathologic_t = pz.Field(desc="The AJCC pathologic T", required=False)
    ajcc_pathologic_n = pz.Field(desc="The AJCC pathologic N", required=False)
    ajcc_pathologic_stage = pz.Field(desc="The AJCC pathologic stage", required=False)
    tumor_grade = pz.Field(desc="The tumor grade", required=False)
    tumor_focality = pz.Field(desc="The tumor focality", required=False)
    tumor_largest_dimension_diameter = pz.Field(desc="The largest...", required=False)
    primary_diagnosis = pz.Field(desc="The primary diagnosis", required=False)
    morphology = pz.Field(desc="The morphology", required=False)
    tissue_or_organ_of_origin = pz.Field(desc="The tissue or organ...", required=False)
    filename = pz.Field(desc="The name of the file...", required=False)
    study = pz.Field(desc="The last name of the author of the study...", required=False)


# define logical plan
xls = pz.Dataset("medical-eval", schema=pz.XLSFile)
patient_tables = xls.convert(
    pz.Table, desc="All tables in the file", cardinality="oneToMany"
)
patient_tables = patient_tables.filter("The rows of the table contain the patient age")
case_data = patient_tables.convert(
    CaseData, desc="The patient data in the table", cardinality="oneToMany"
)

# create and execute physical plans...
```

Figure 10: The AI program written using PALIMPZEST for the Medical Schema Matching workload.